Østfold University College

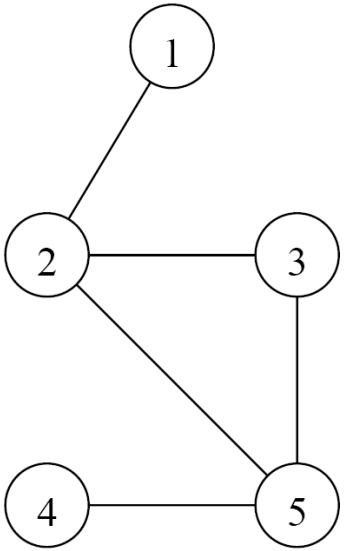
AI Graph Search Algorithms

Østfold University College

Graph- Data structure (Reminder)

Adjacency matrix

The space required to store the adjacency matrix of a graph with n vertices is $\Theta(n^2)$.



$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

Figure 10.2 An undirected graph and its adjacency matrix representation.

Adjacency list

The space required to store the adjacency list is $\Theta(|E|)$.

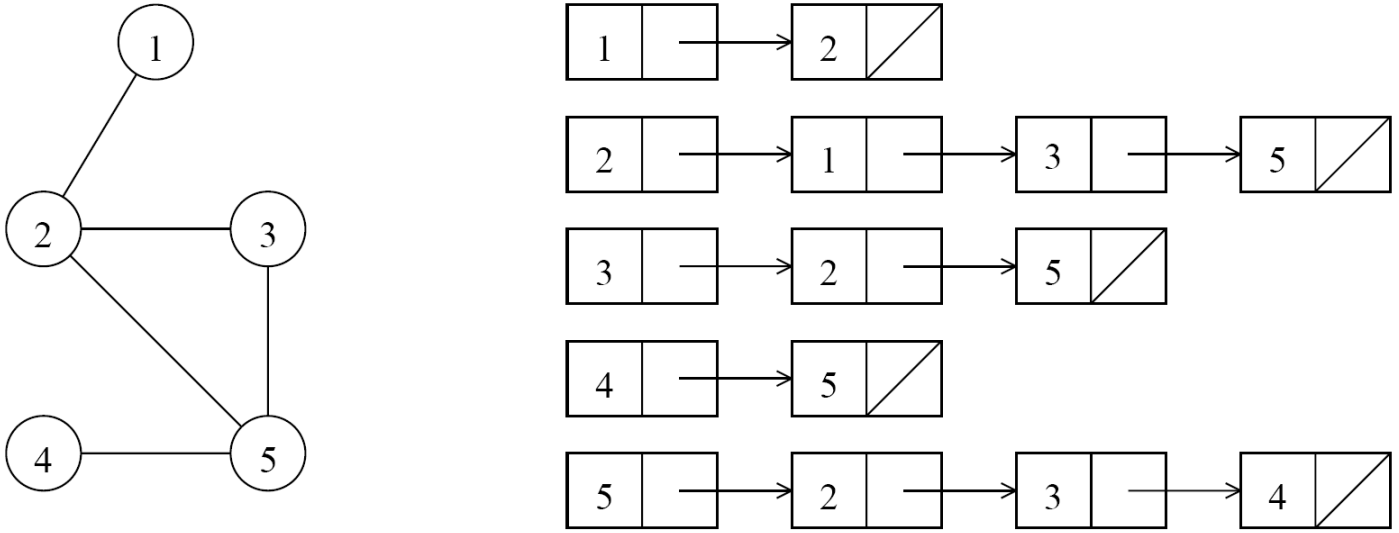
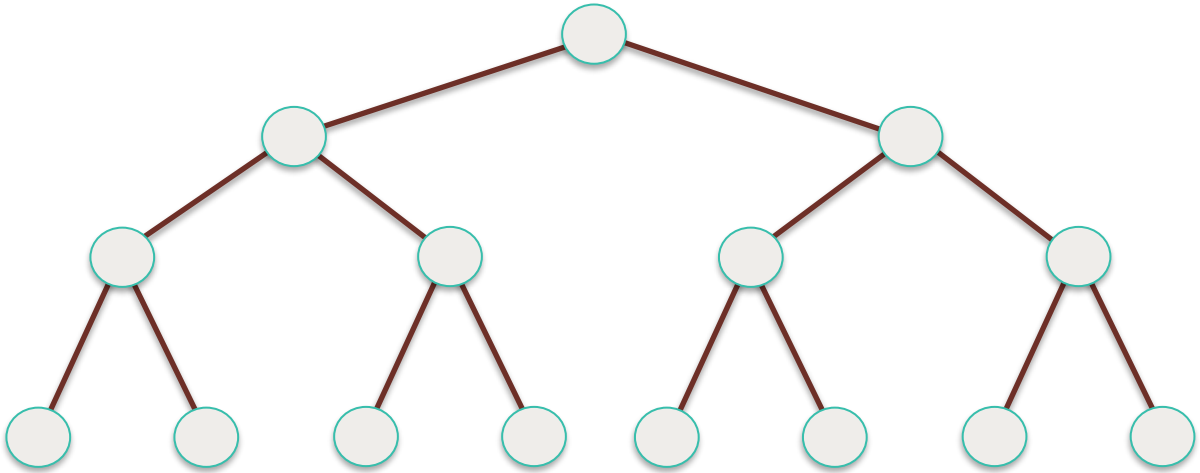


Figure 10.3 An undirected graph and its adjacency list representation.

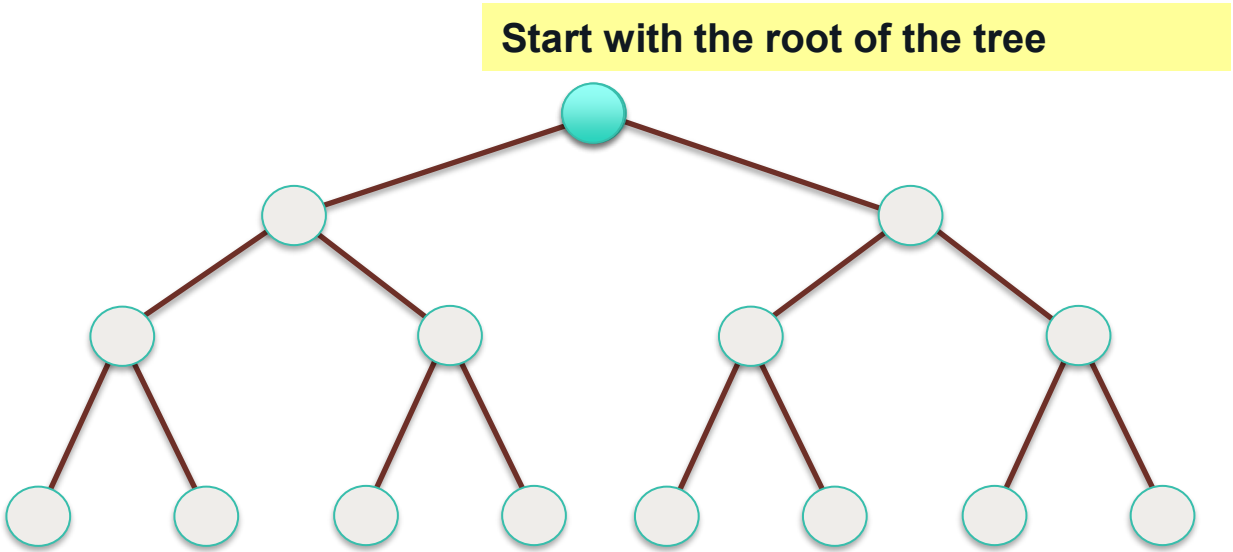
Østfold University College

Serial Breadth-First Search (BFS)

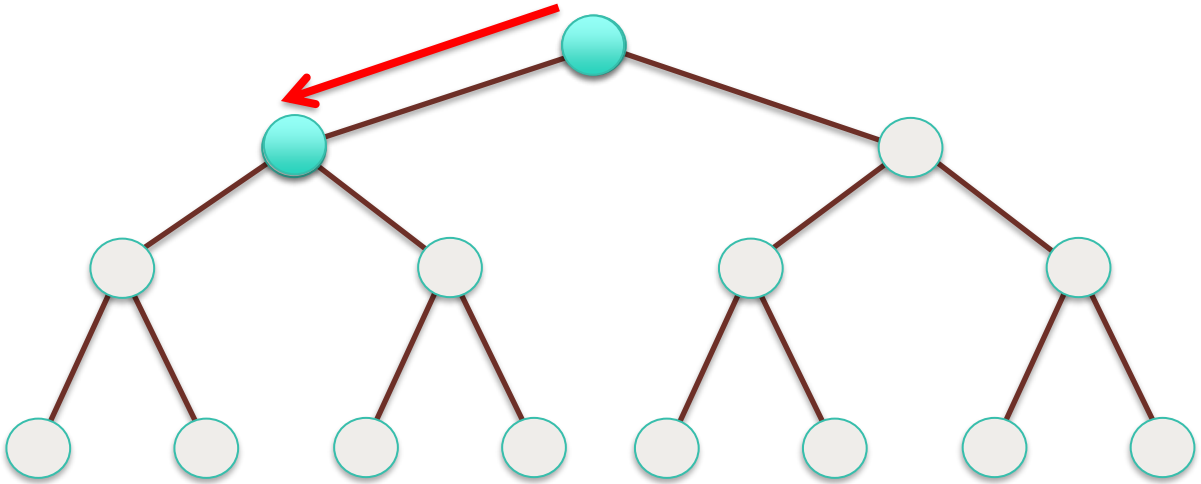
Breadth-First Search (BFS): Tree



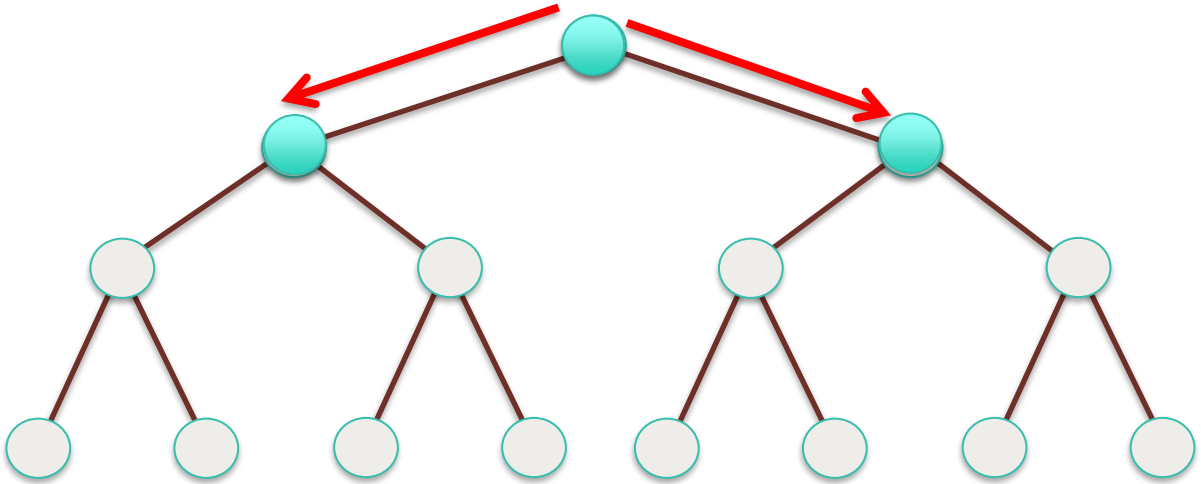
Breadth-First Search (BFS): Tree



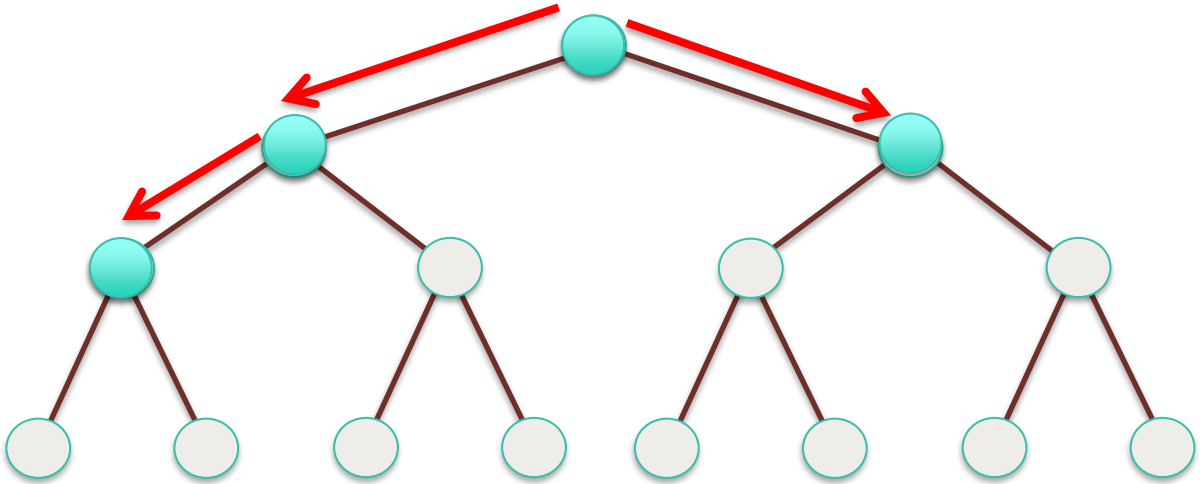
Breadth-First Search (BFS): Tree



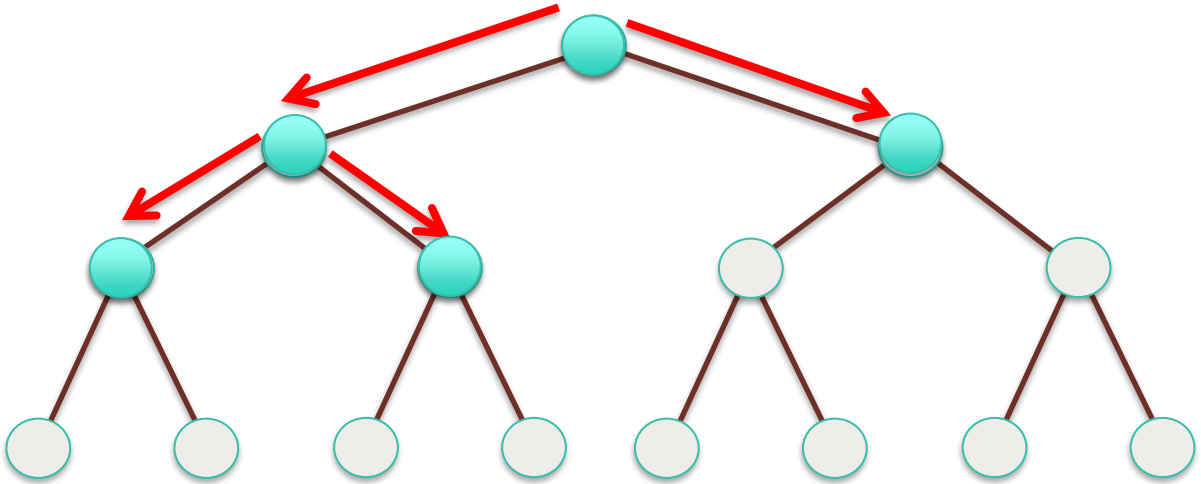
Breadth-First Search (BFS): Tree



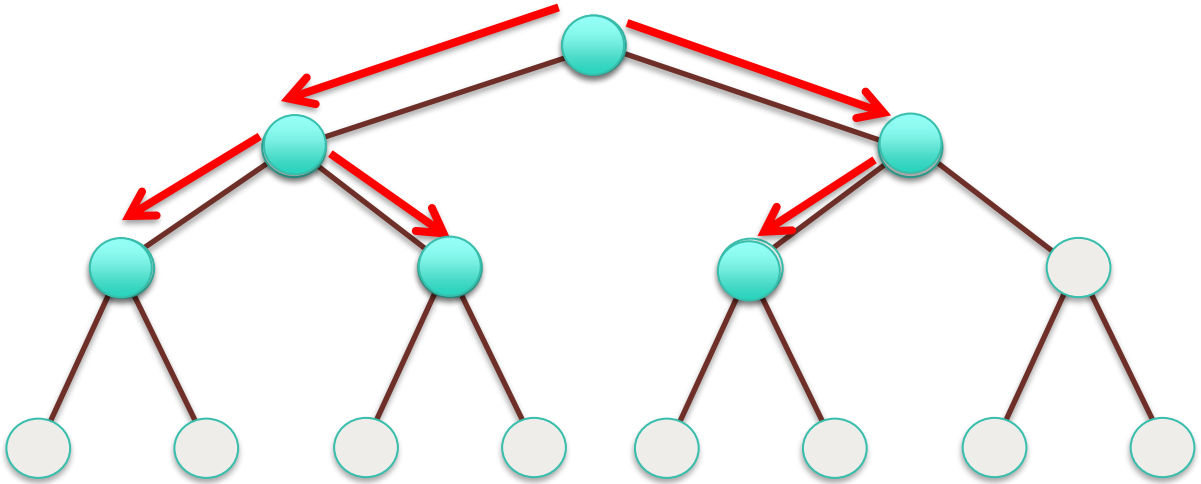
Breadth-First Search (BFS): Tree



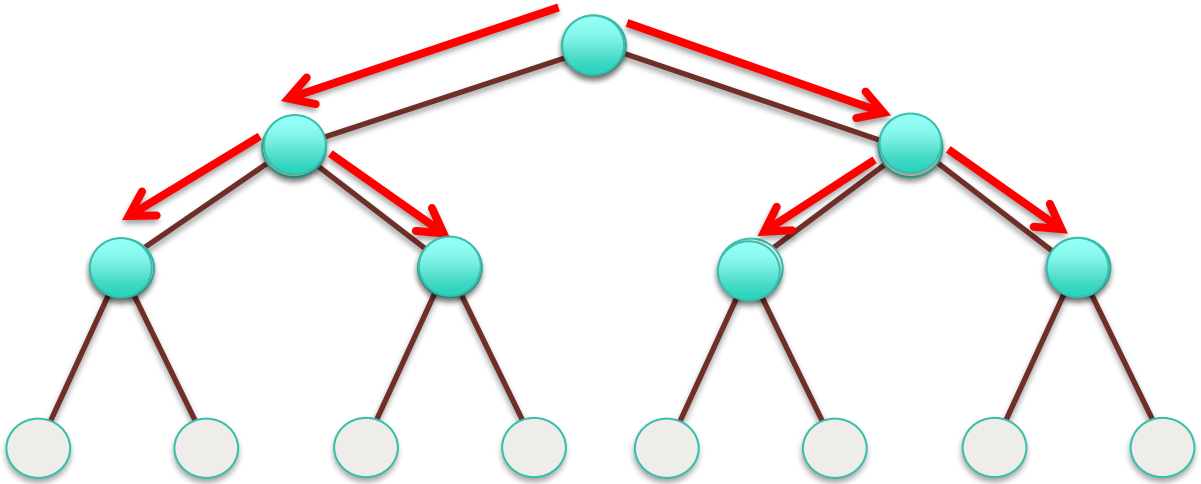
Breadth-First Search (BFS): Tree



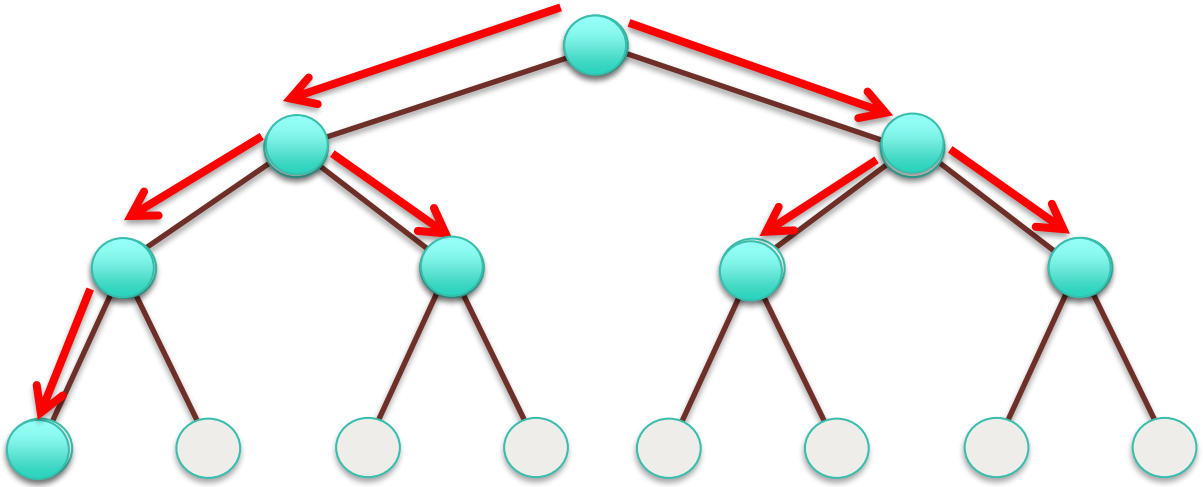
Breadth-First Search (BFS): Tree



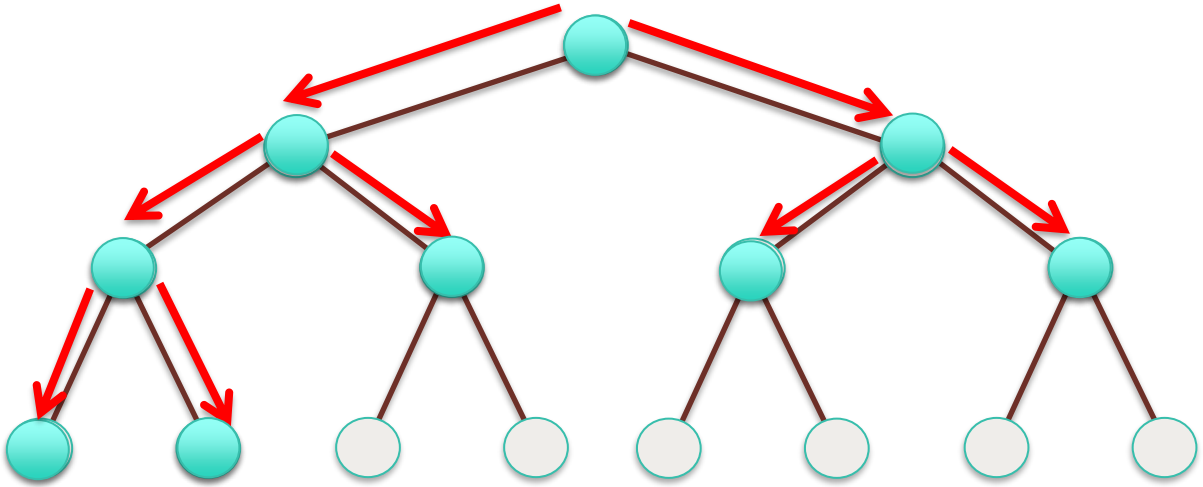
Breadth-First Search (BFS): Tree



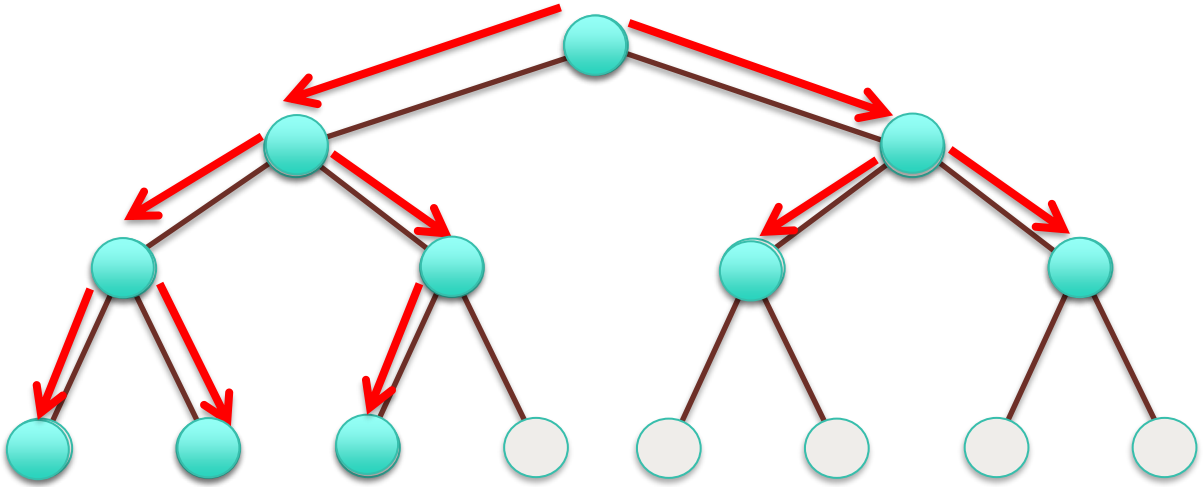
Breadth-First Search (BFS): Tree



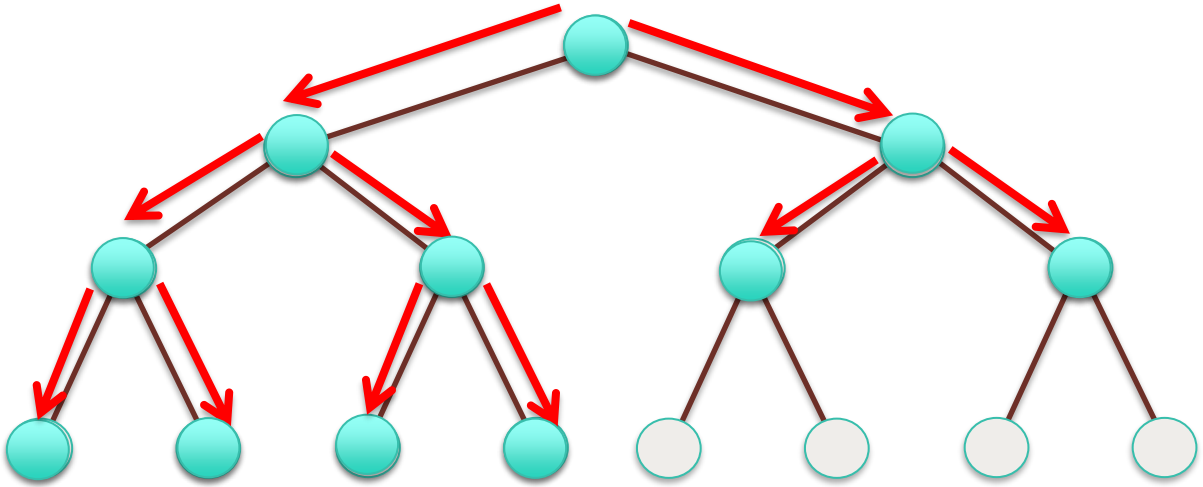
Breadth-First Search (BFS): Tree



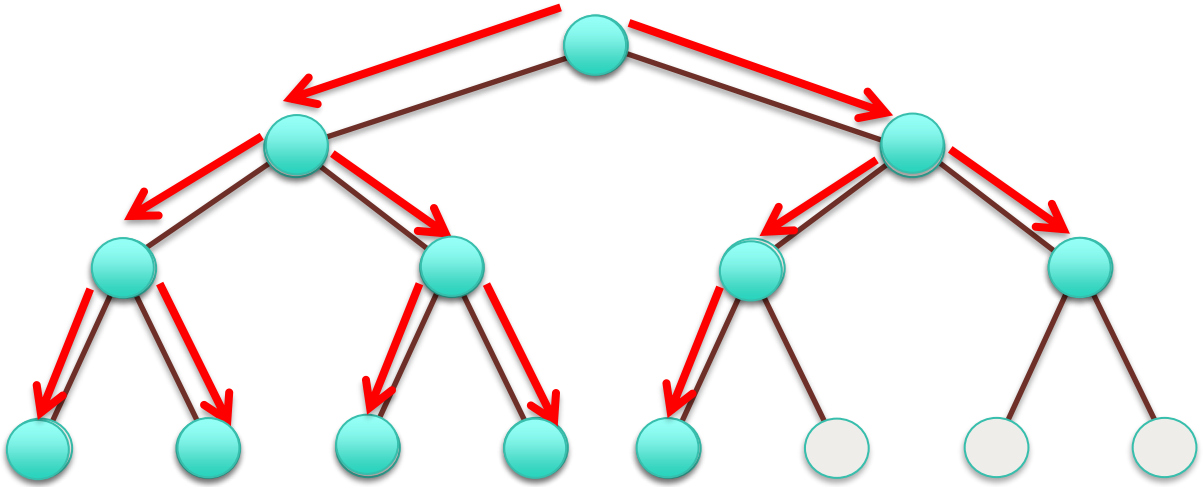
Breadth-First Search (BFS): Tree



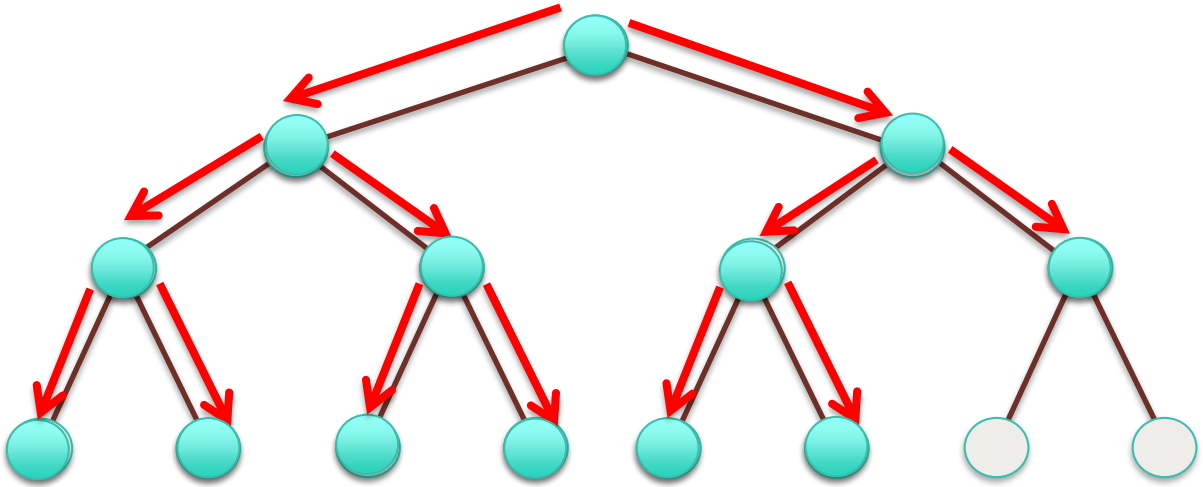
Breadth-First Search (BFS): Tree



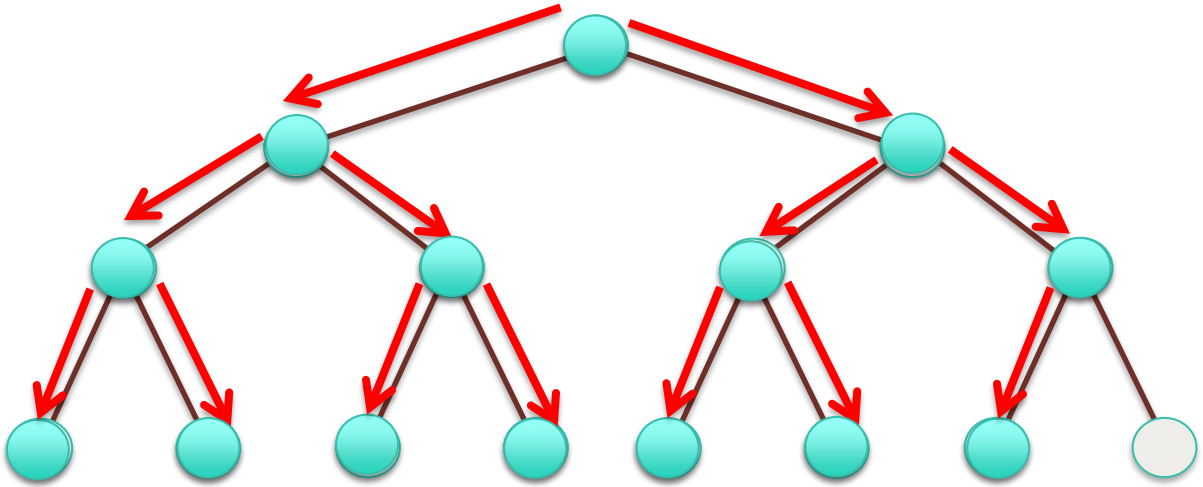
Breadth-First Search (BFS): Tree



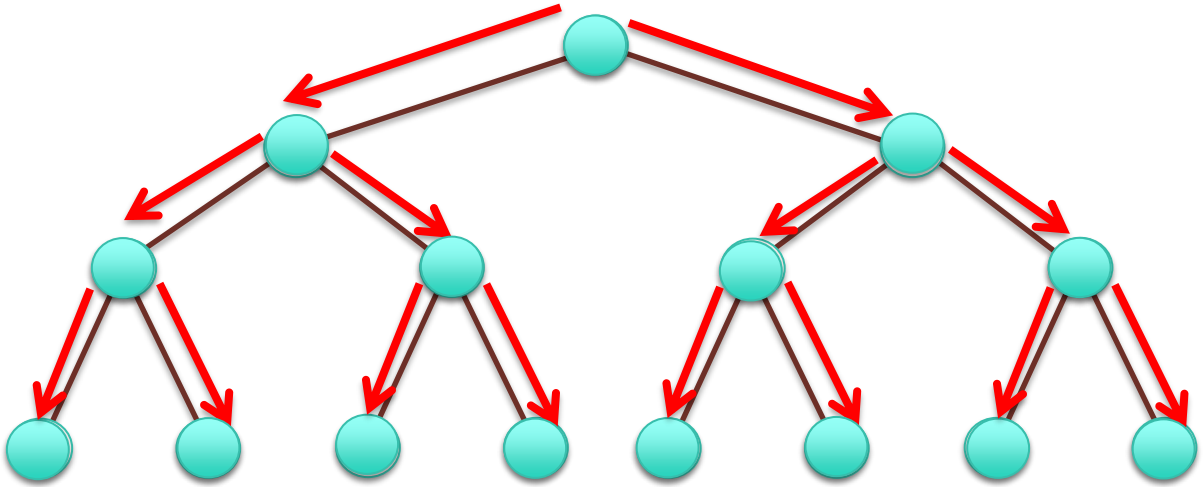
Breadth-First Search (BFS): Tree



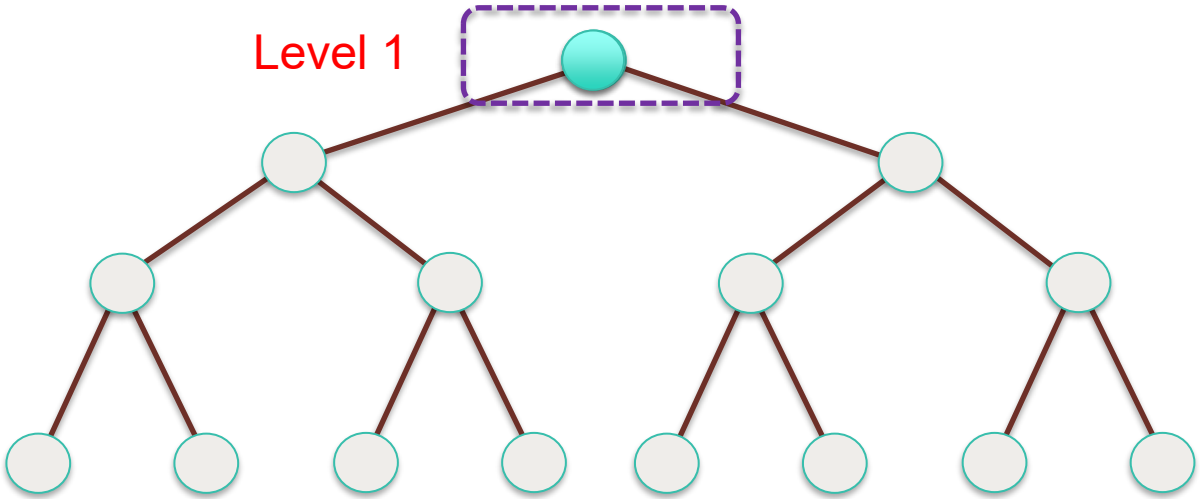
Breadth-First Search (BFS): Tree



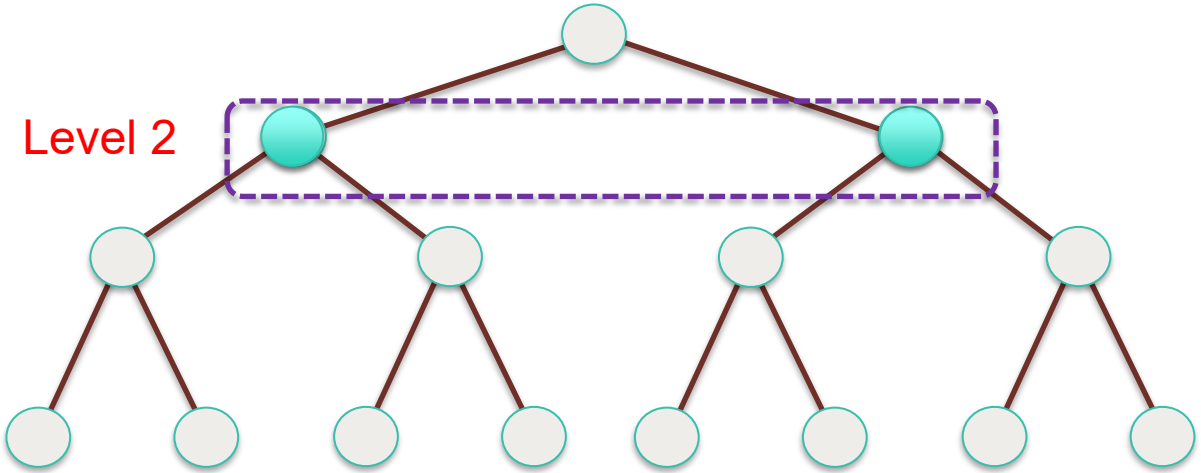
Breadth-First Search (BFS): Tree



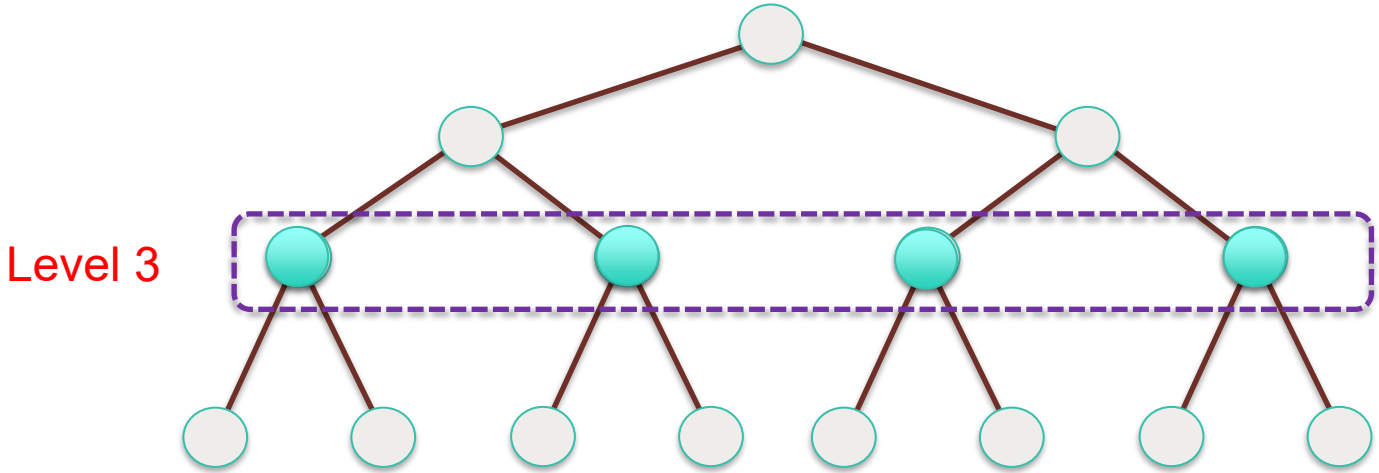
Breadth-First Search (BFS): Tree



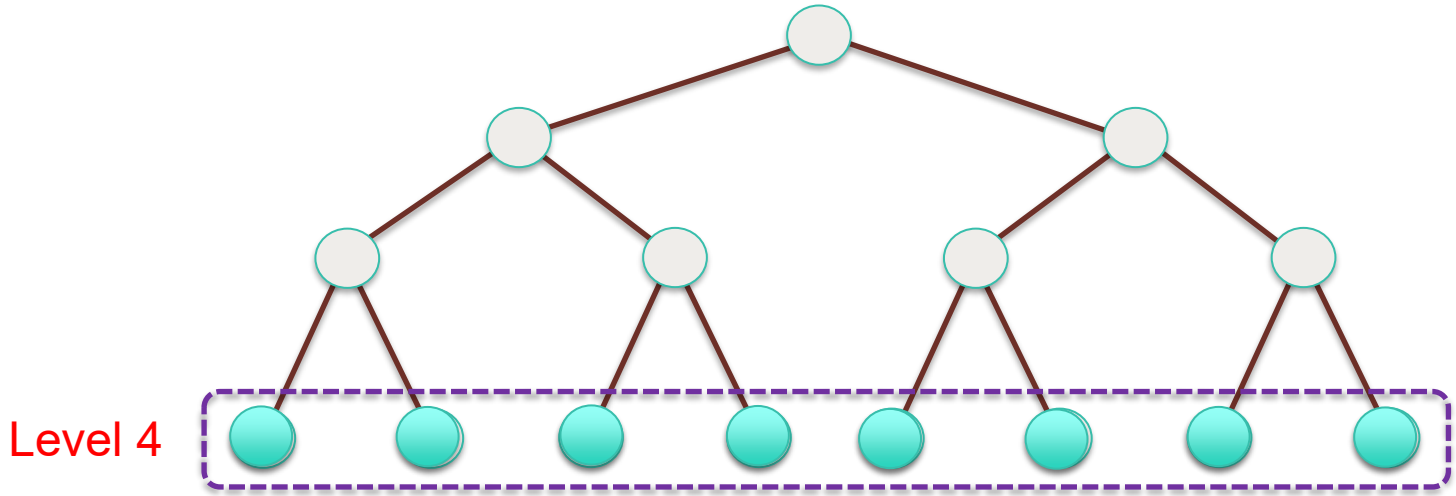
Breadth-First Search (BFS): Tree



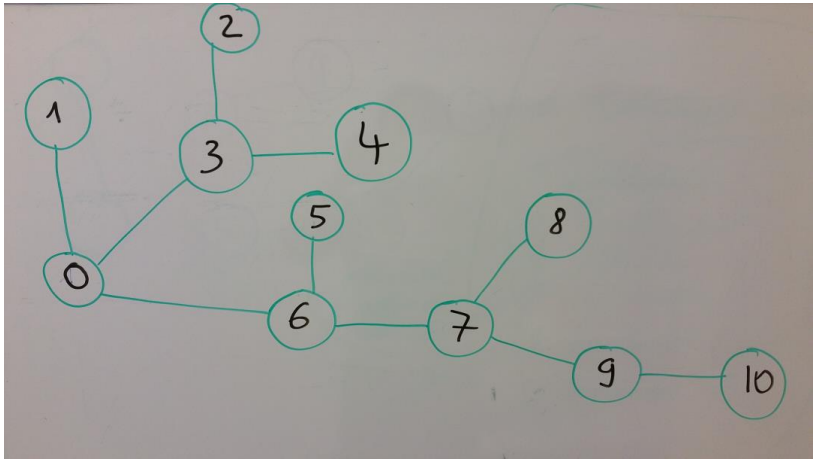
Breadth-First Search (BFS): Tree



Breadth-First Search (BFS): Tree

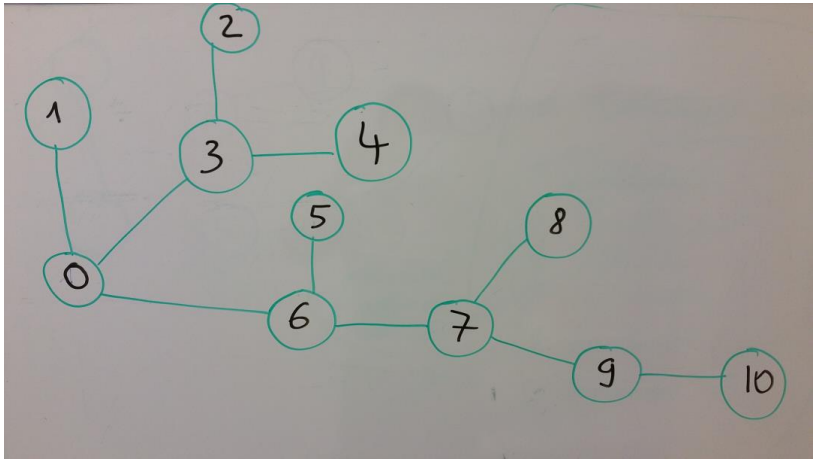


Idea for Implementation



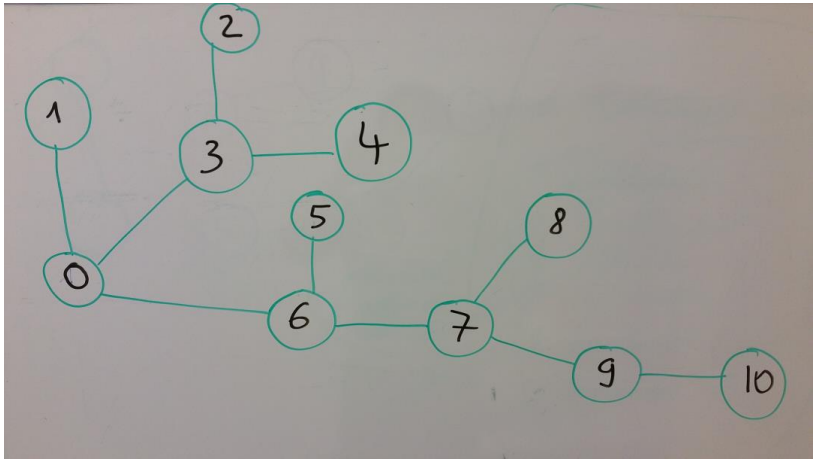
Data structure first: adjacent matrix or adjacent list ?
Start vertex: 0 (but can be any other vertex)
Data Structure: Queue (First-in-First-Out)
BFS Visit List = {0,1,3,6,2,4,5,7,8,9,10}
Step 1: Q = {0} => Pick 0 and visit 0. Add 0 to visit list
Step 2: Q = {1,3,6} => Pick 1 and add 1 to visit list
Step 3: Q = {3,6} => Pick 3. Add 3 to visit list and children of 3 to Q.
Step 4: Q = {6,2,4} =>Pick 6. Add 6 to visit list and children of 6 to Q.
Step 6: Q = {2,4,5,7} =>Pick 2. Add 2 to visit list.
Step 7: Q= {4,5,7} =>Pick 4. Add 4 to visit list.
Step 8: Q= {5,7} =>Pick 5. Add 5 to visit list.
Step 9: Q= {7} => Pick 7. Add 7 to visit list. Then add children of 7 to Q.
Step 10: Q= {8,9} => Pick 8. Add 8 to visit list.
Step 11: Q= {9} => Pick 9. Add 9 to visit list and children of 9 to Q.
Step 12: Q= {10} => Pick 10. Add 10 to visit list .
Step 13: Q= {∅} **STOP**

Idea for Implementation



Graph Search: Maintain list of explored nodes

Idea for Implementation

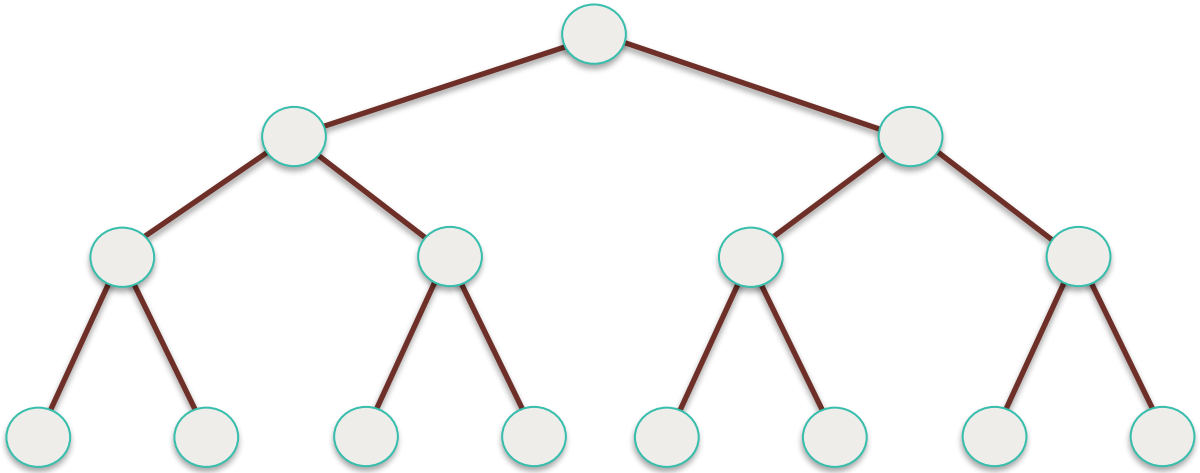


Data structure first: adjacent matrix or adjacent list ?
Start vertex: 0 (but can be any other vertex)
Data Structure: Queue (First-in-First-Out)
BFS Visit List = ?
Step 1: Q = {0}
Step 2: Q = {1,3,6}
Step 3: Q = {3,6}
Step 4: Q = {6,2,4}
Step 5: Q = {6,2,4}
Step 6: Q = {2,4,5,7}
Step 7: Q= {4,5,7}
Step 9: Q= {5,7}
Step 10: Q= {7}
Step 11: Q= {8,9}
Step 12: Q= {9}
Step 13: Q= {10}
Step 14: Q= {∅} **STOP**

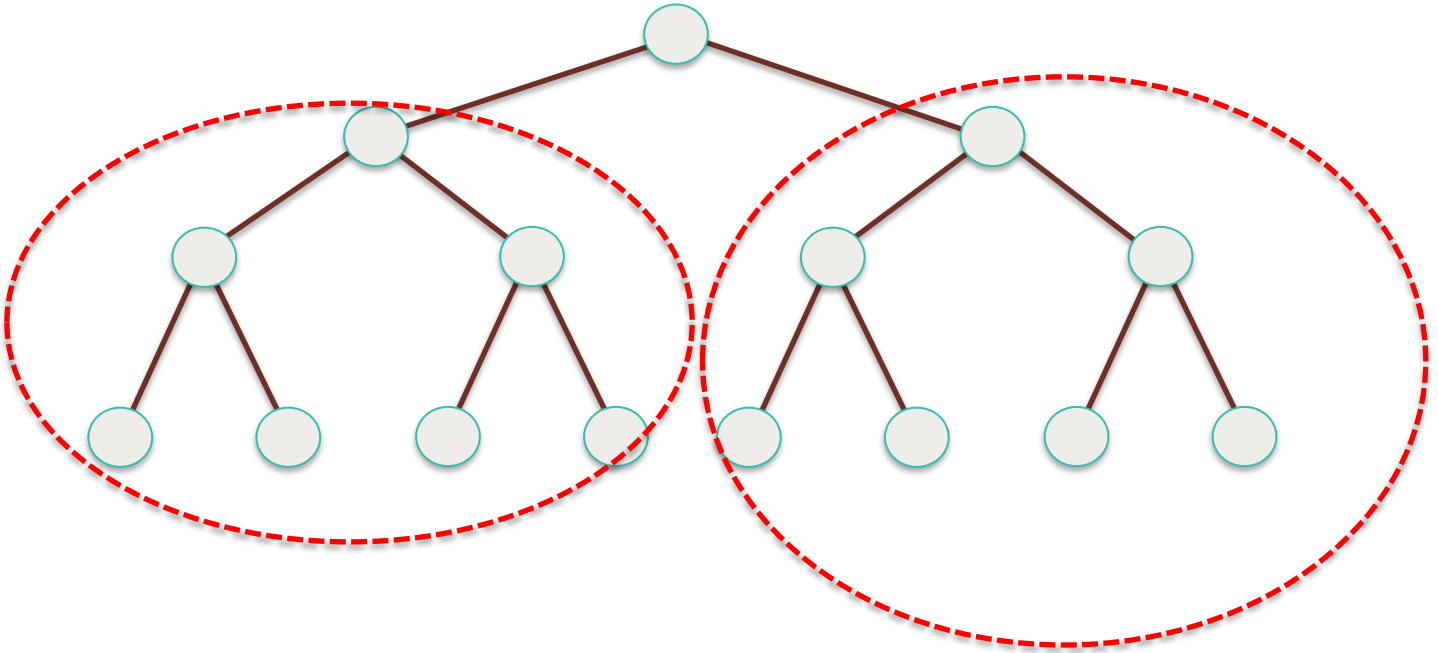
Use add(), addAll(), poll() methods in ConcurrentLinkedQueue

Parallelize BFS

Breadth-First Search (BFS): Tree



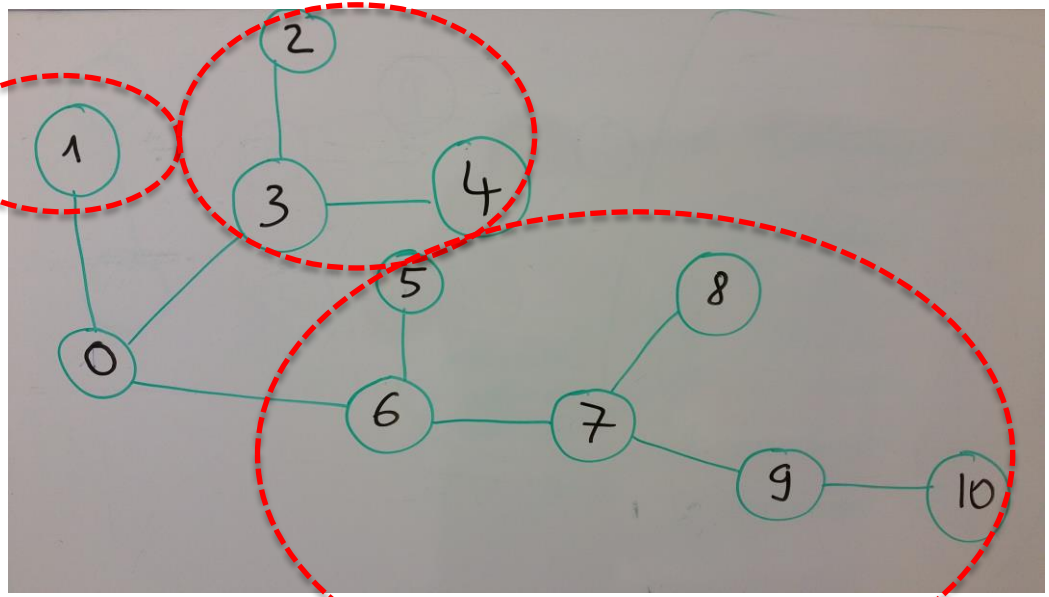
Breadth-First Search (BFS): Tree



BSF

task

task



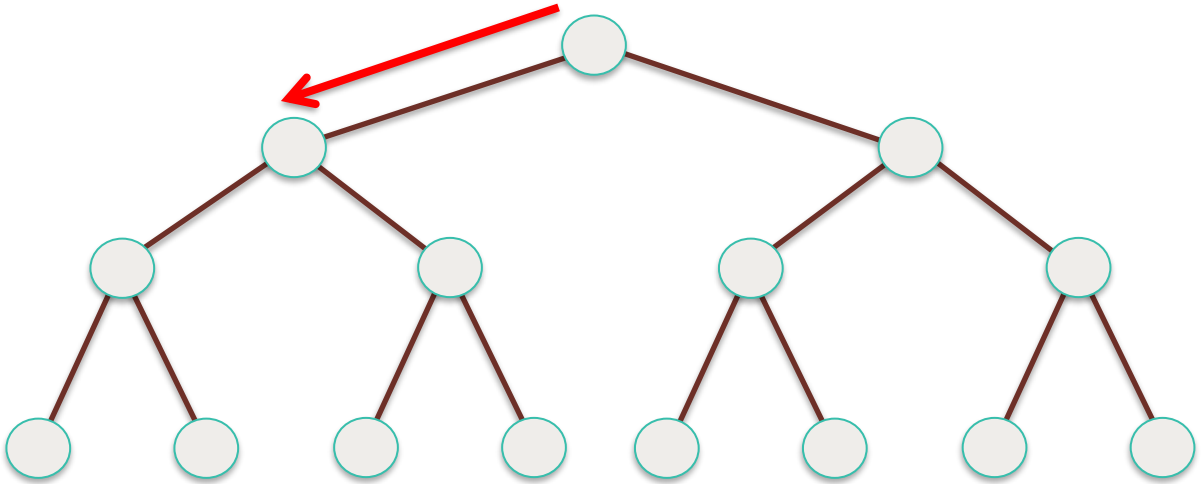
task

Submit tasks to an Executor framework

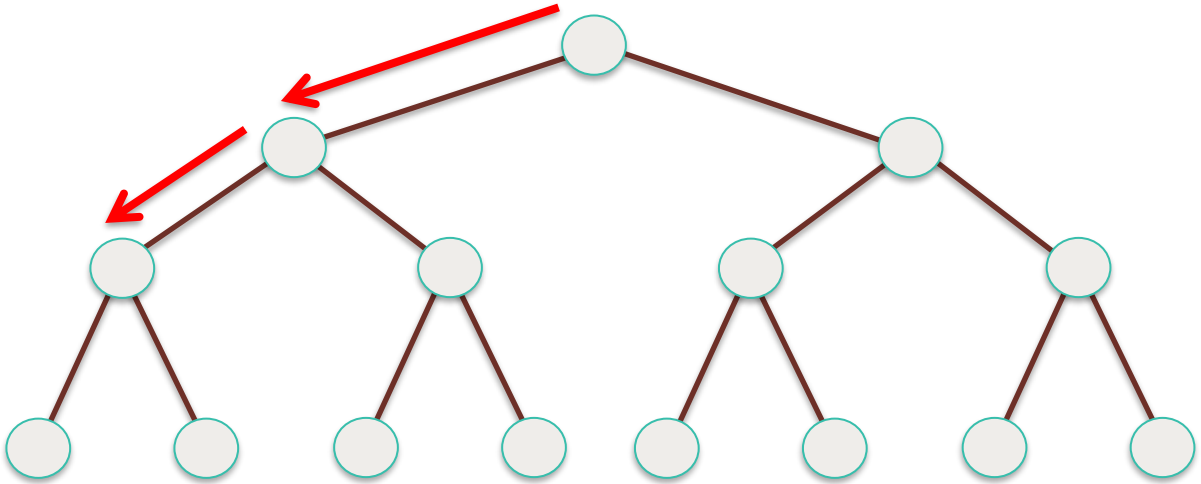
Østfold University College

Depth-First Search (DFS) Algorithm

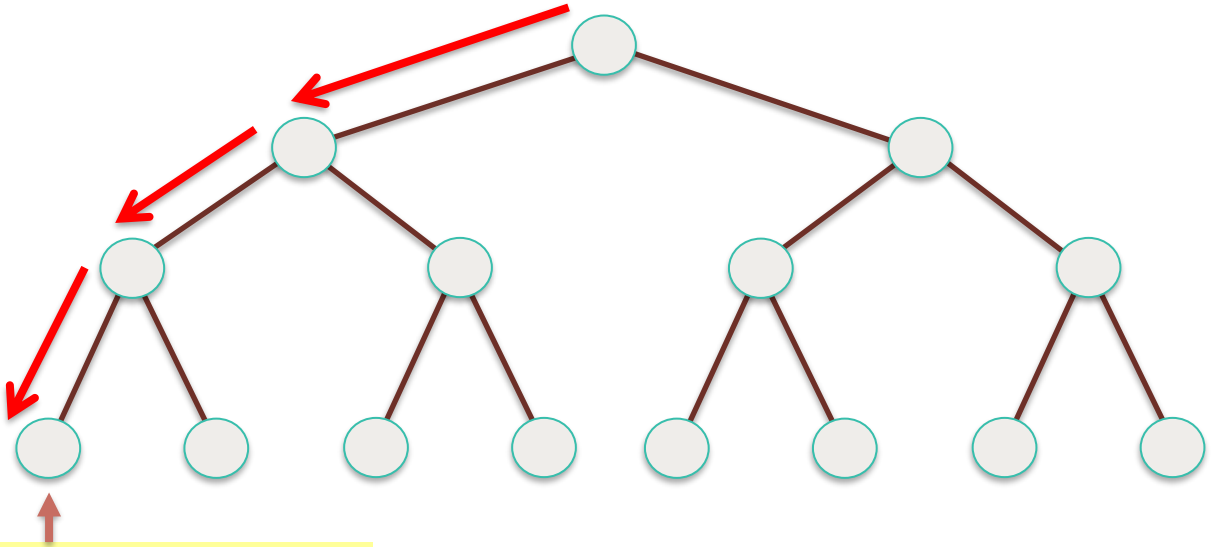
Depth-First Search (DFS): Tree



Depth-First Search (DFS): Tree

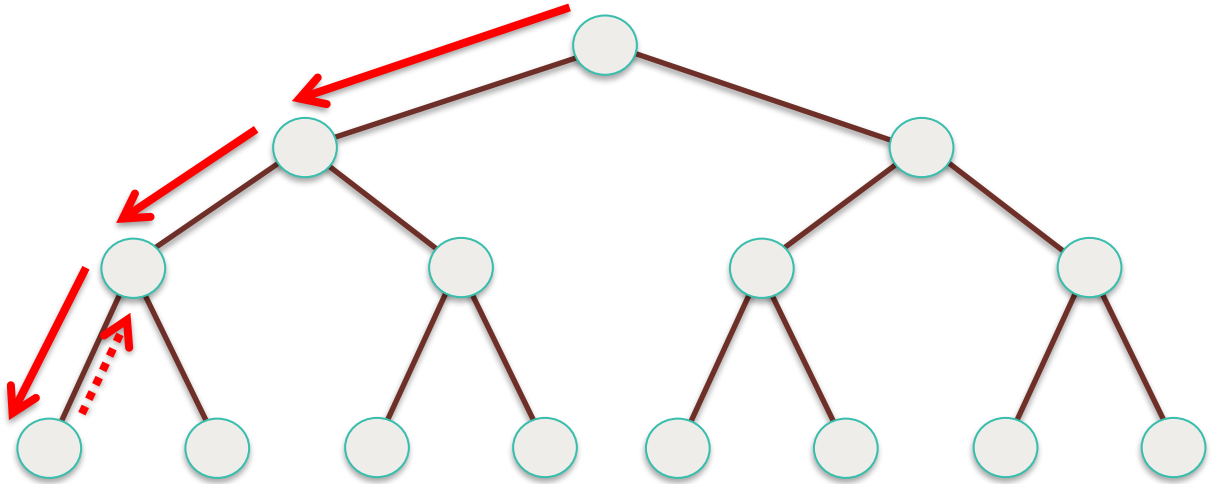


Depth-First Search (DFS): Tree

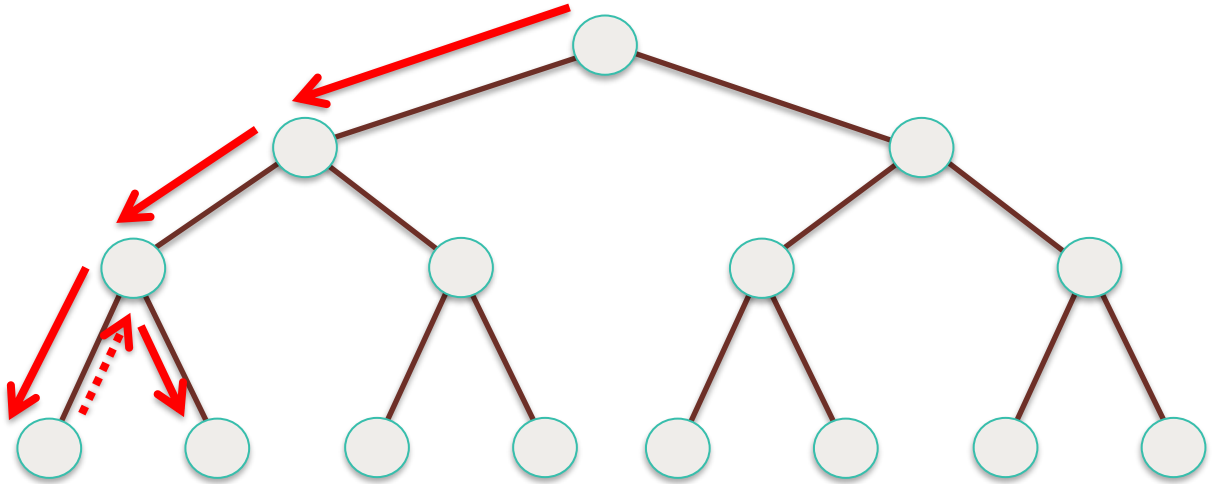


No successor, backtracking!

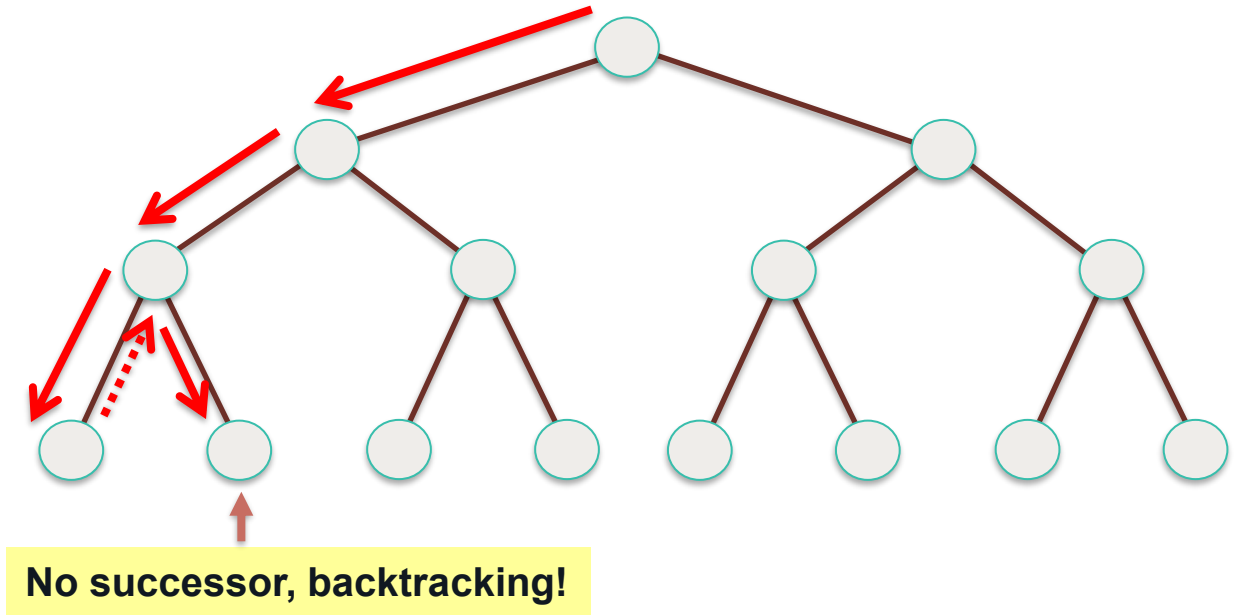
Depth-First Search (DFS): Tree



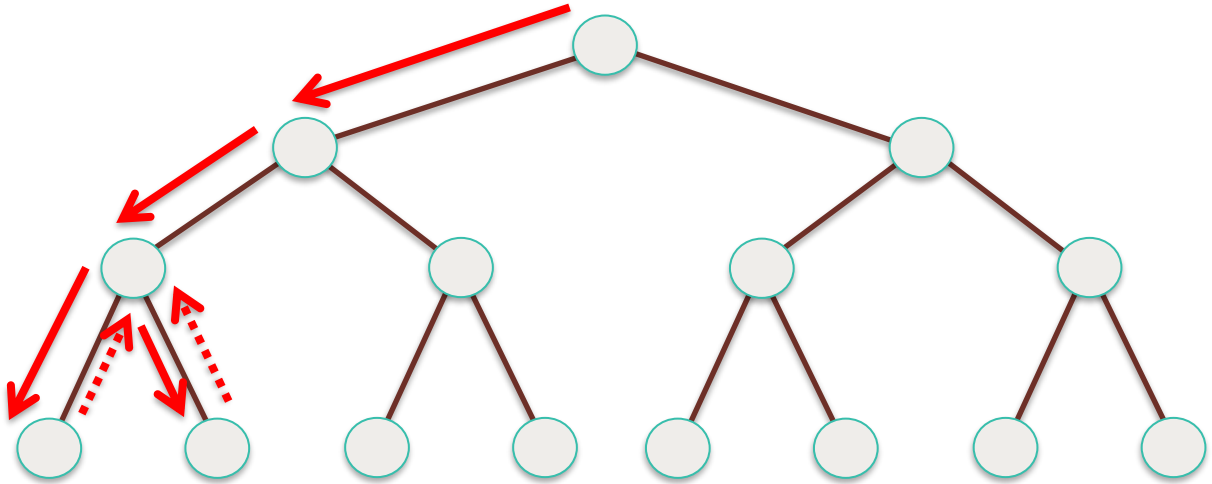
Depth-First Search (DFS): Tree



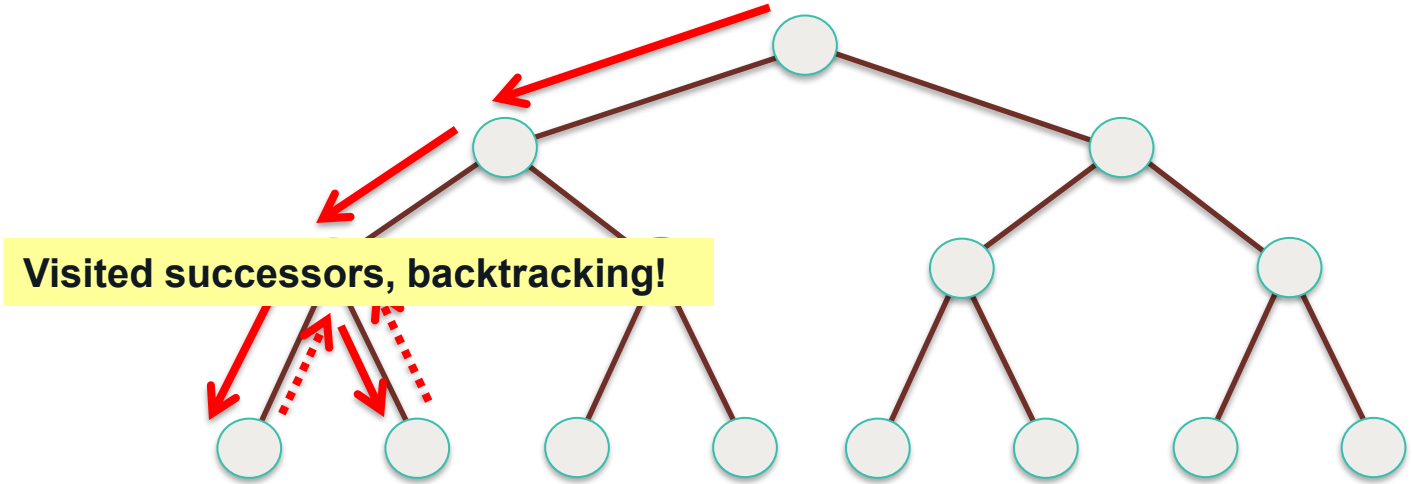
Depth-First Search (DFS): Tree



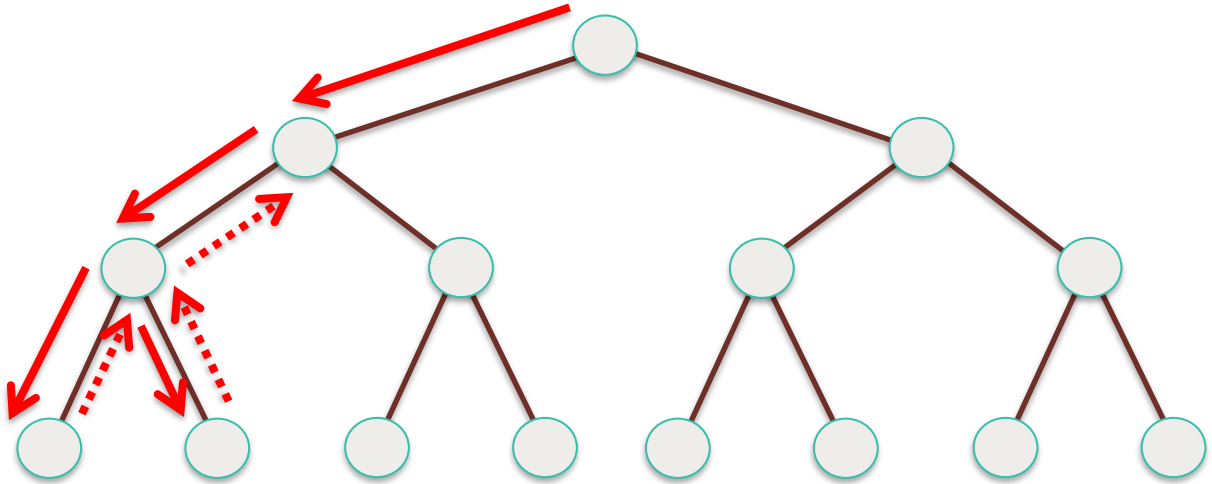
Depth-First Search (DFS): Tree



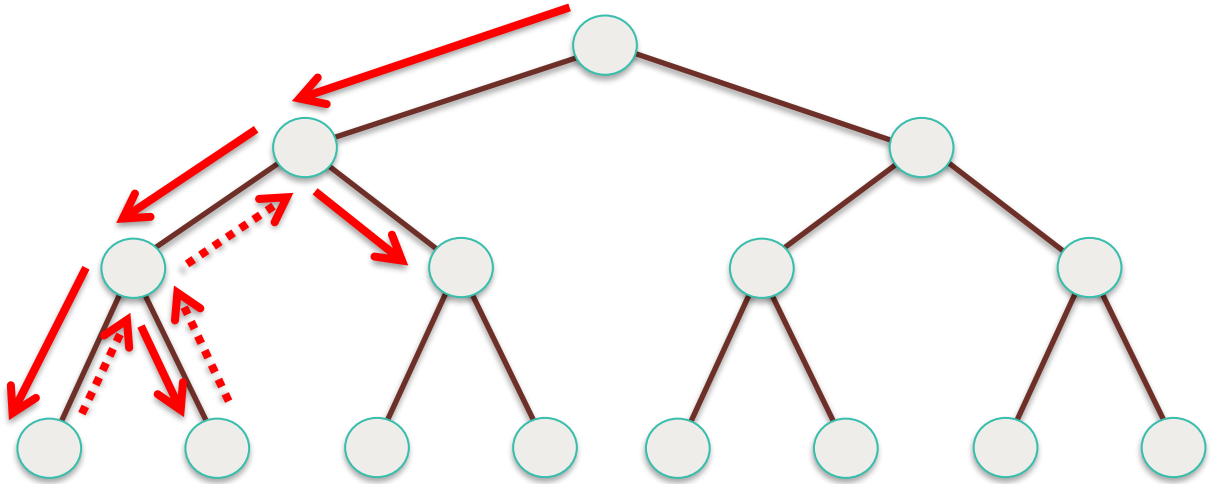
Depth-First Search (DFS): Tree



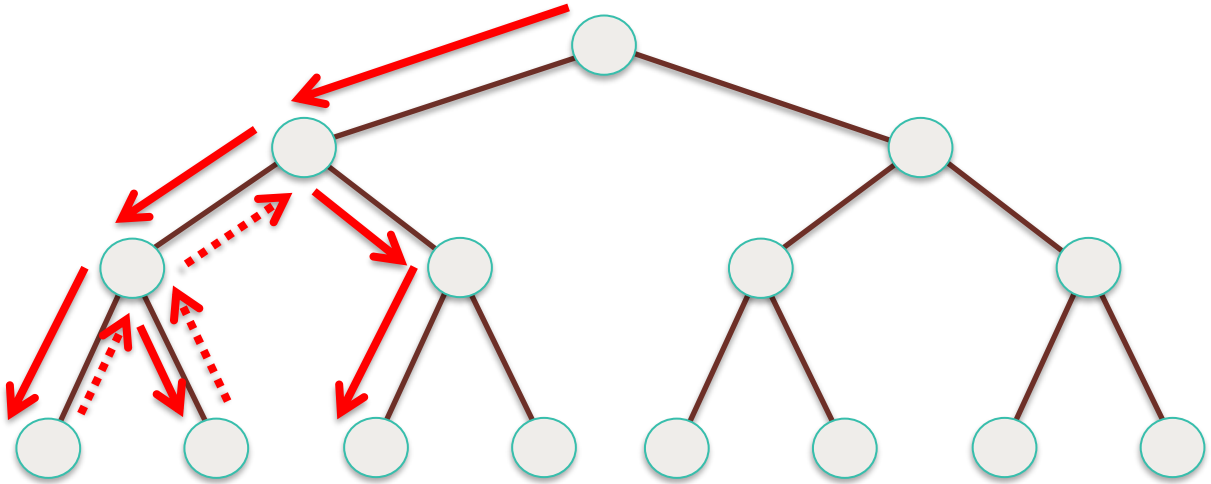
Depth-First Search (DFS): Tree



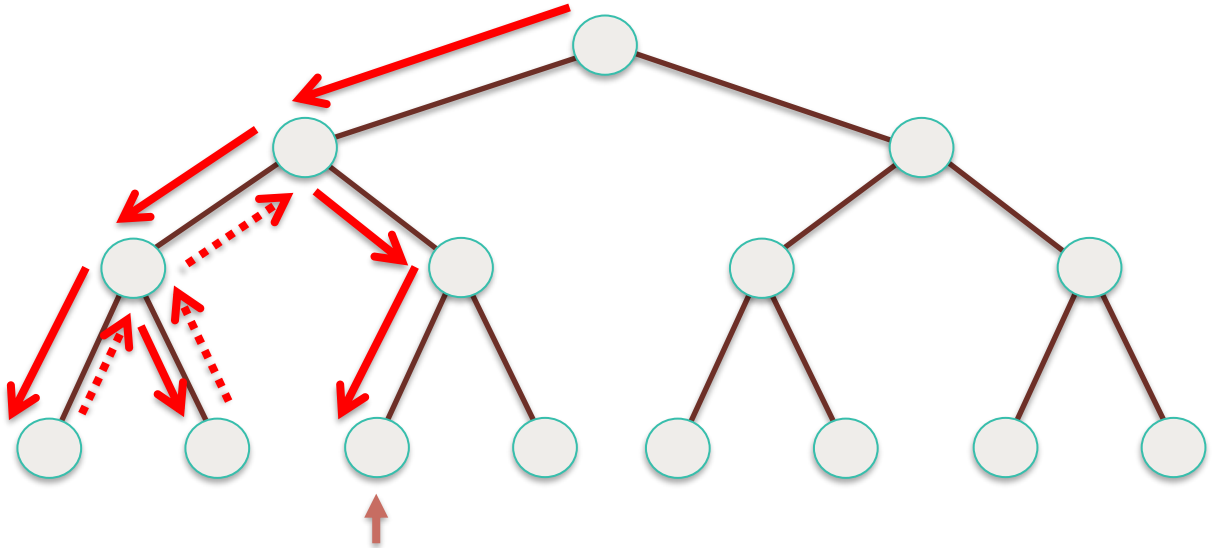
Depth-First Search (DFS): Tree



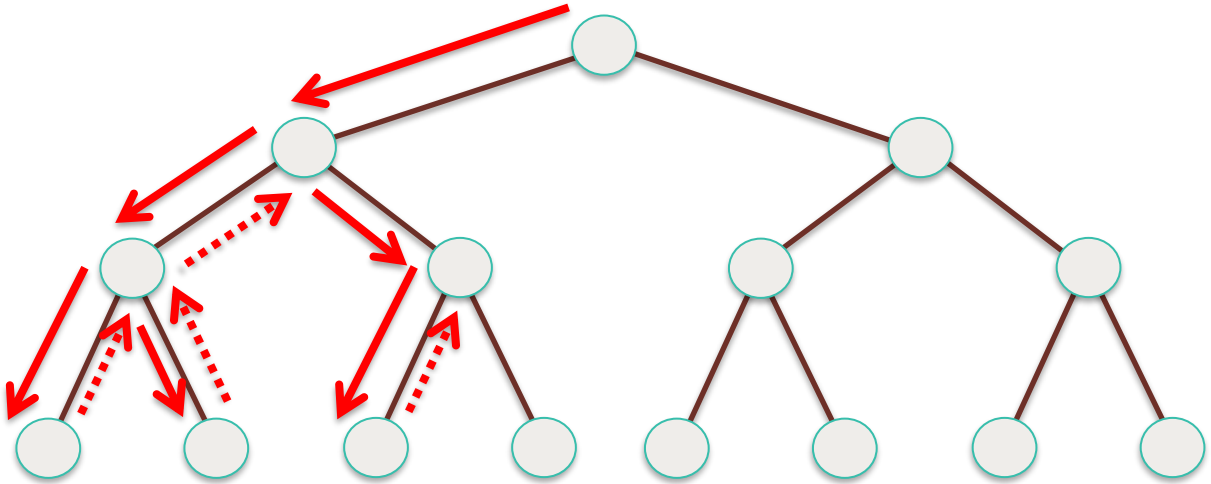
Depth-First Search (DFS): Tree



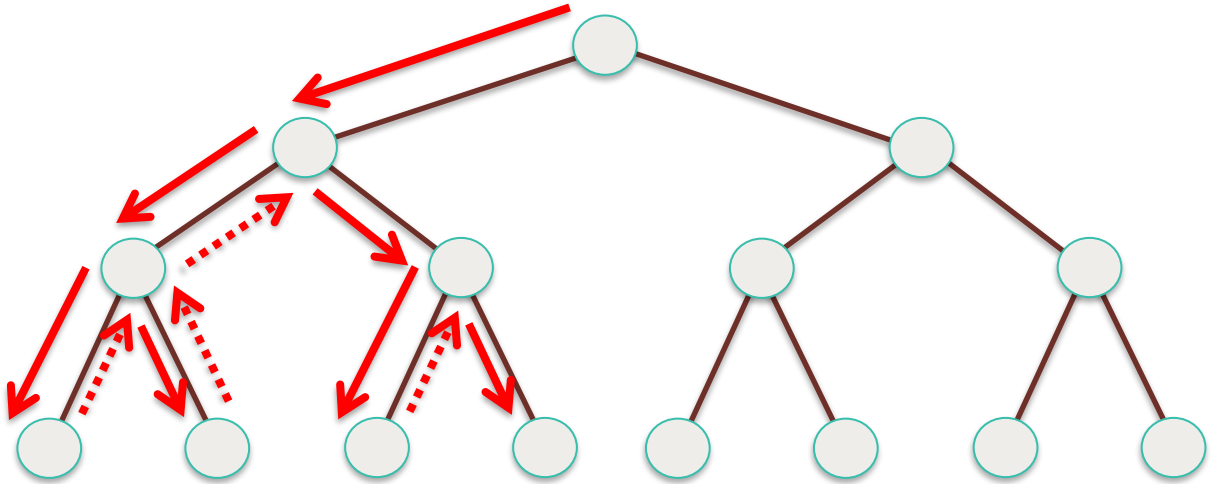
Depth-First Search (DFS): Tree



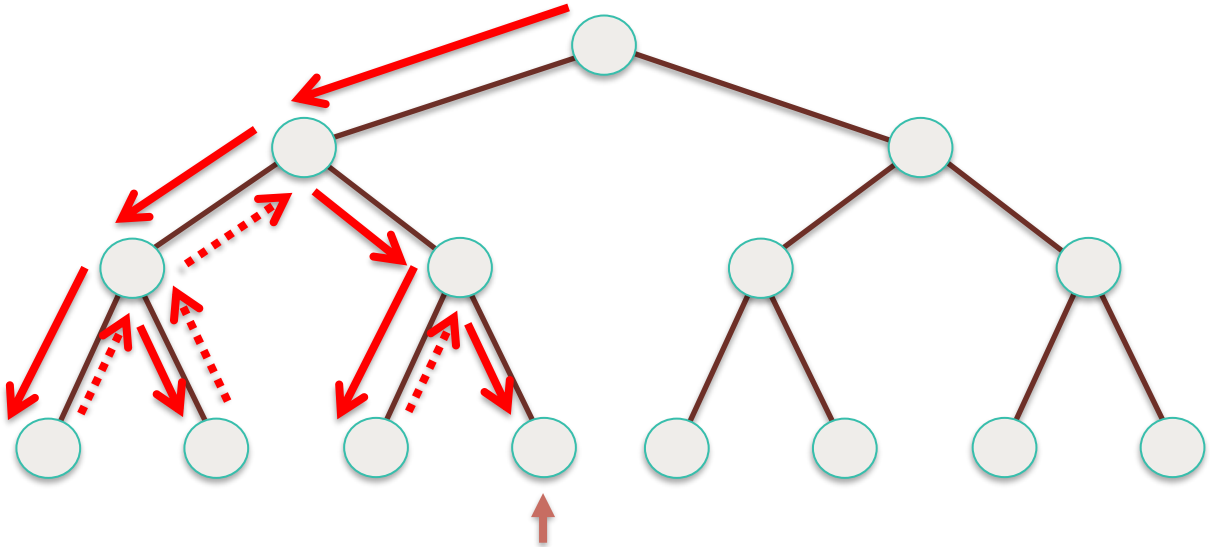
Depth-First Search (DFS): Tree



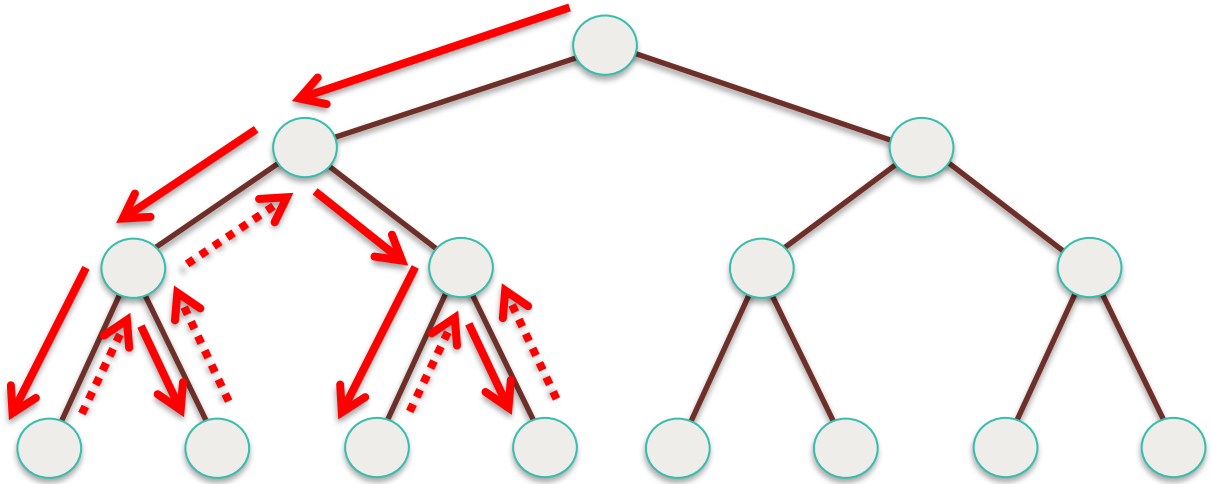
Depth-First Search (DFS): Tree



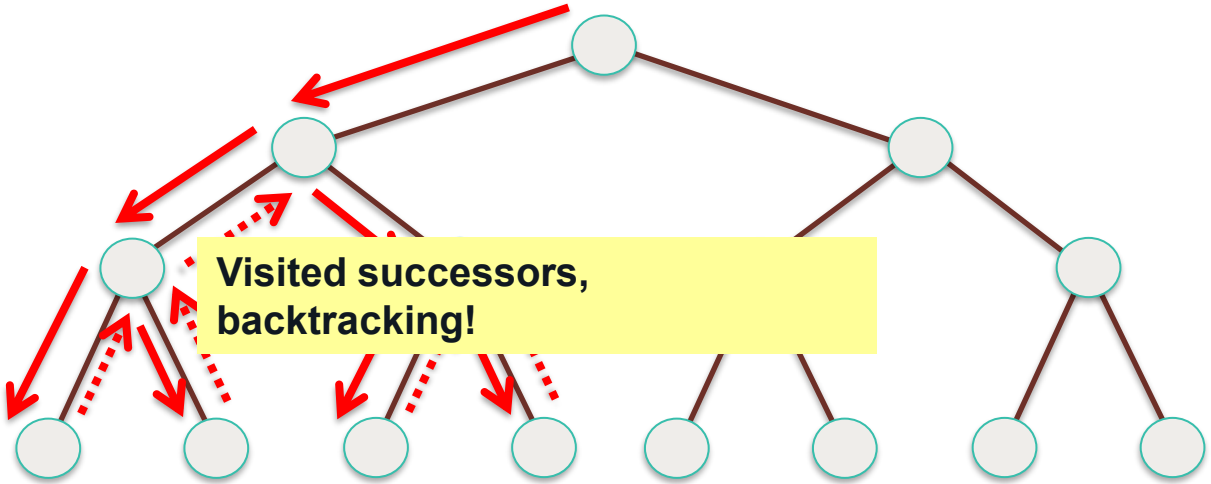
Depth-First Search (DFS): Tree



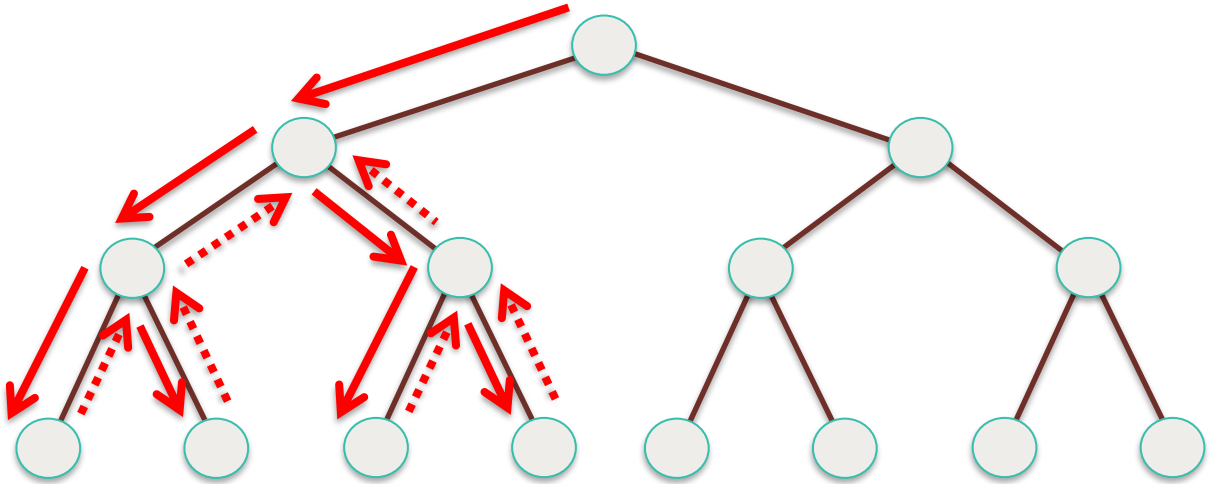
Depth-First Search (DFS): Tree



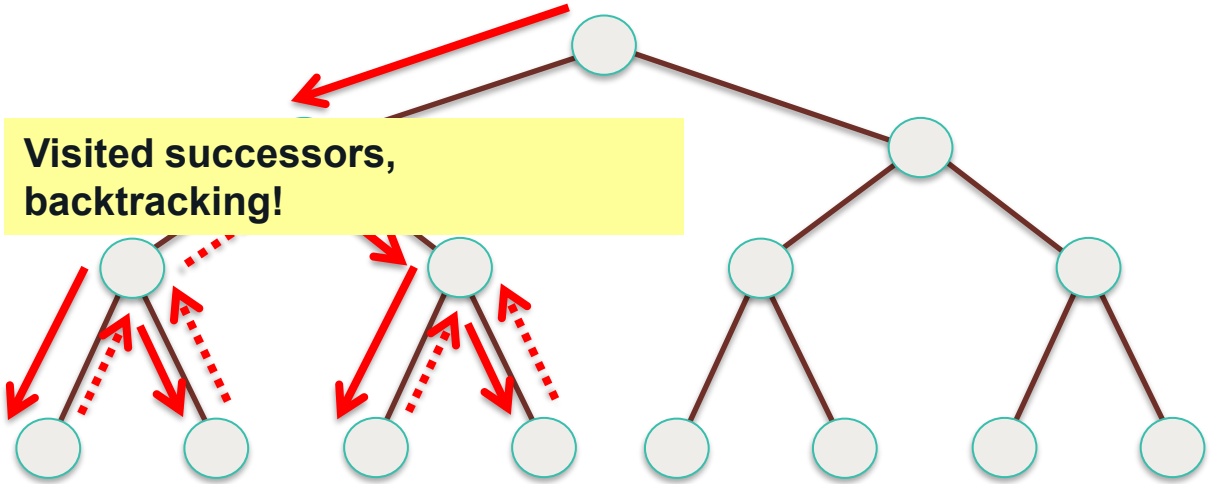
Depth-First Search (DFS): Tree



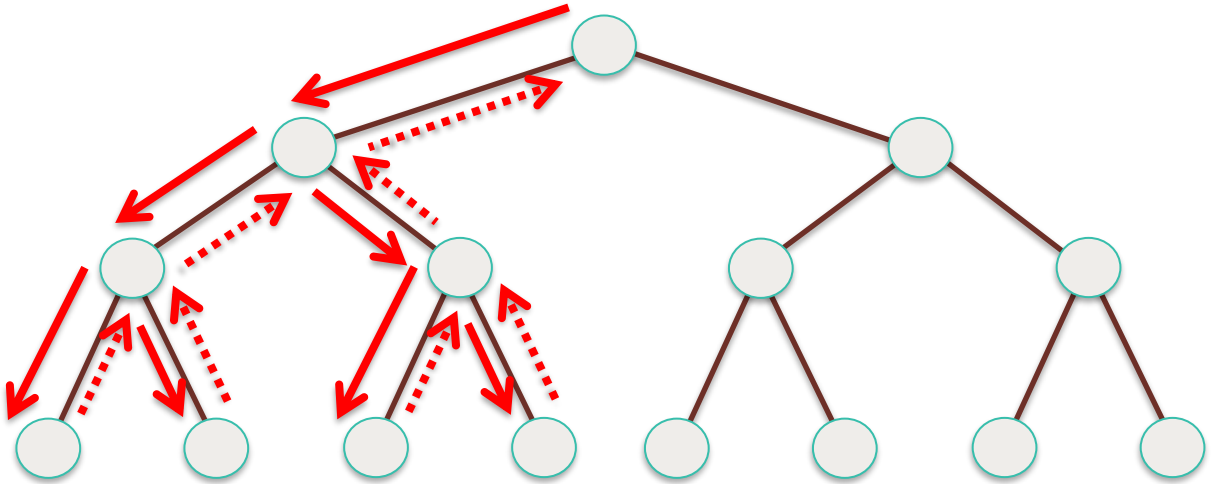
Depth-First Search (DFS): Tree



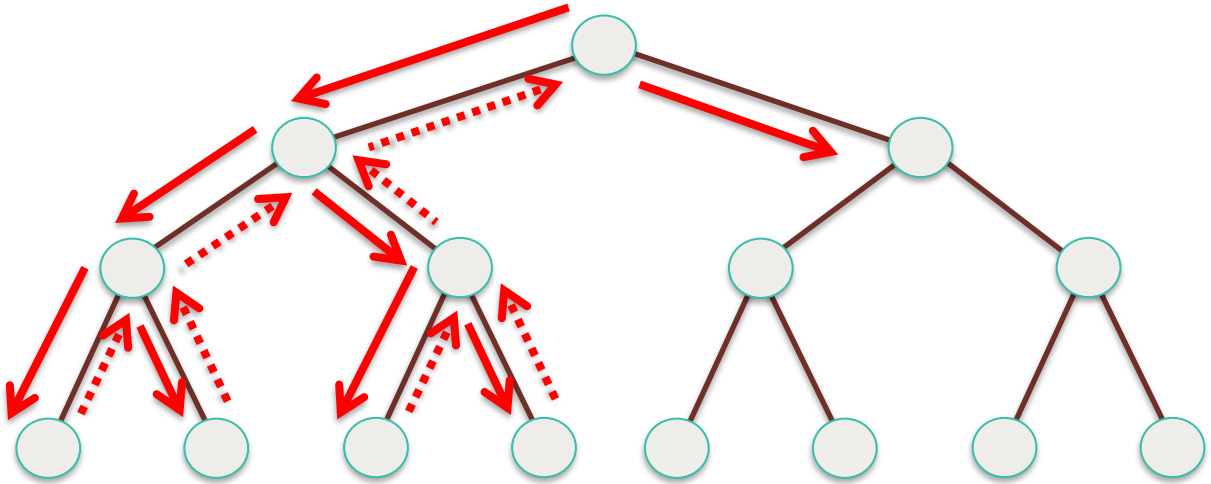
Depth-First Search (DFS): Tree



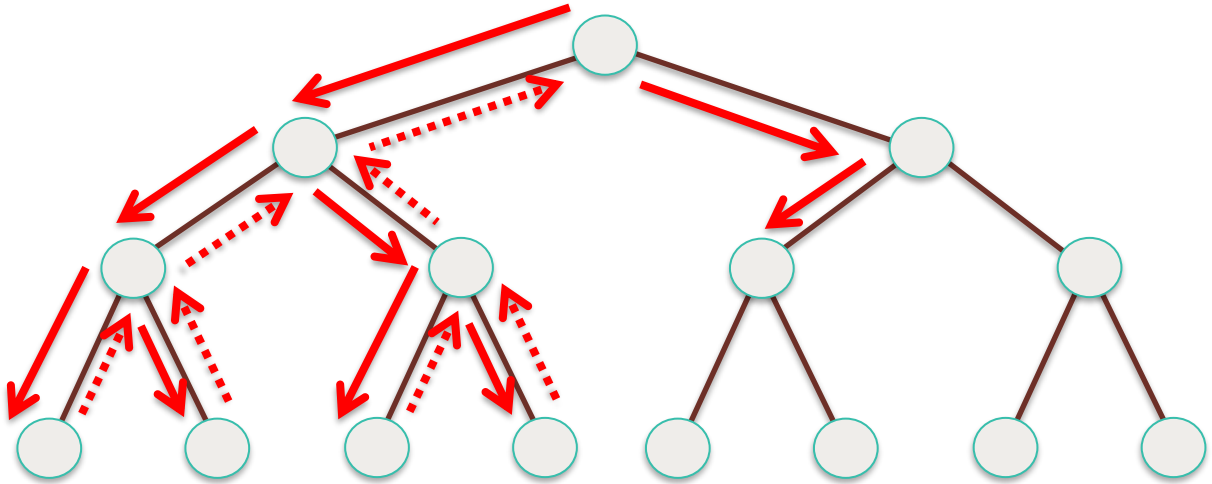
Depth-First Search (DFS): Tree



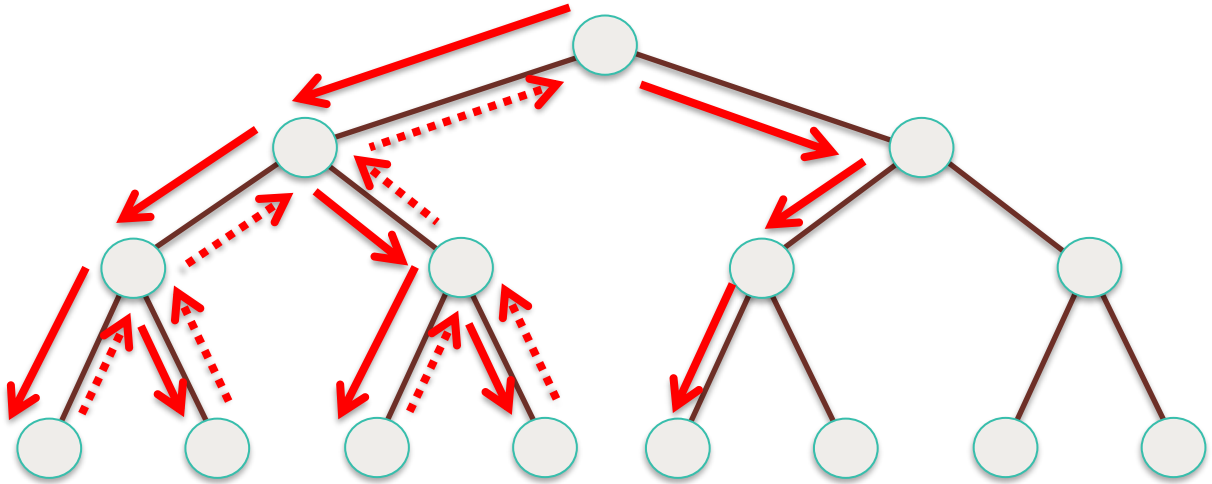
Depth-First Search (DFS): Tree



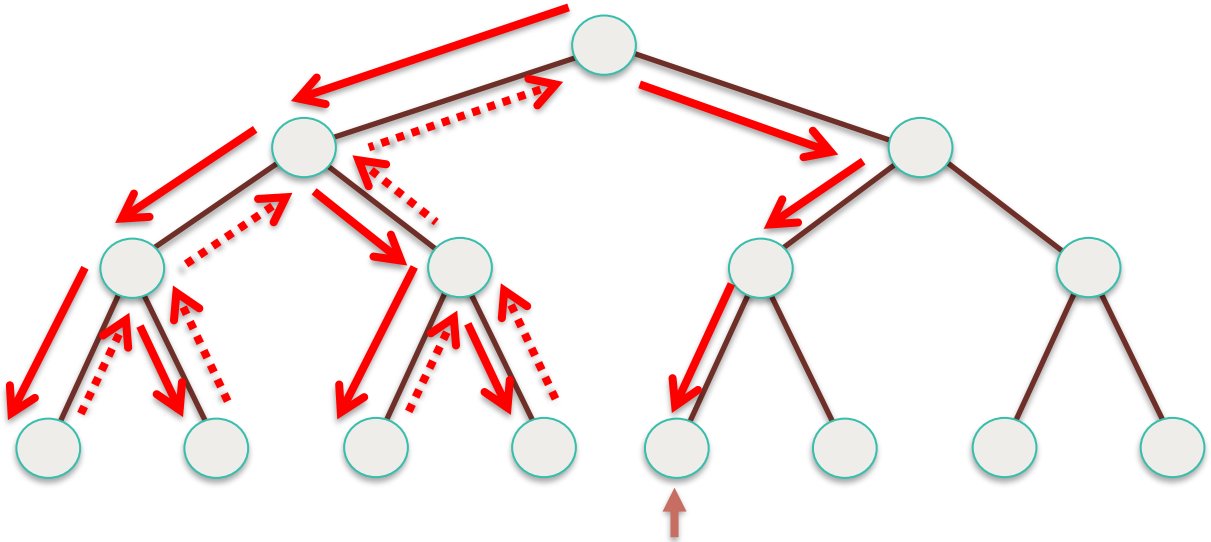
Depth-First Search (DFS): Tree



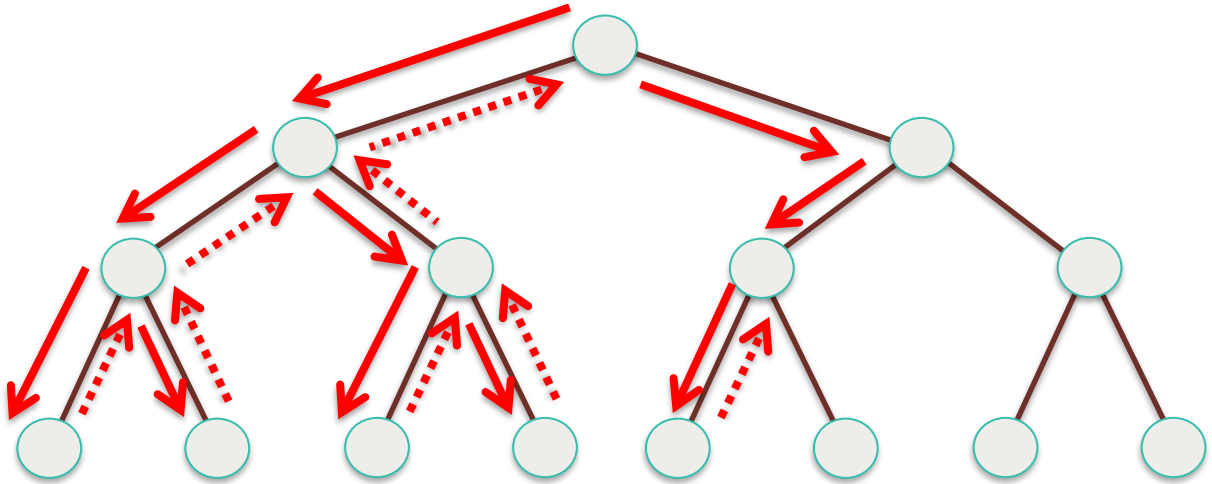
Depth-First Search (DFS): Tree



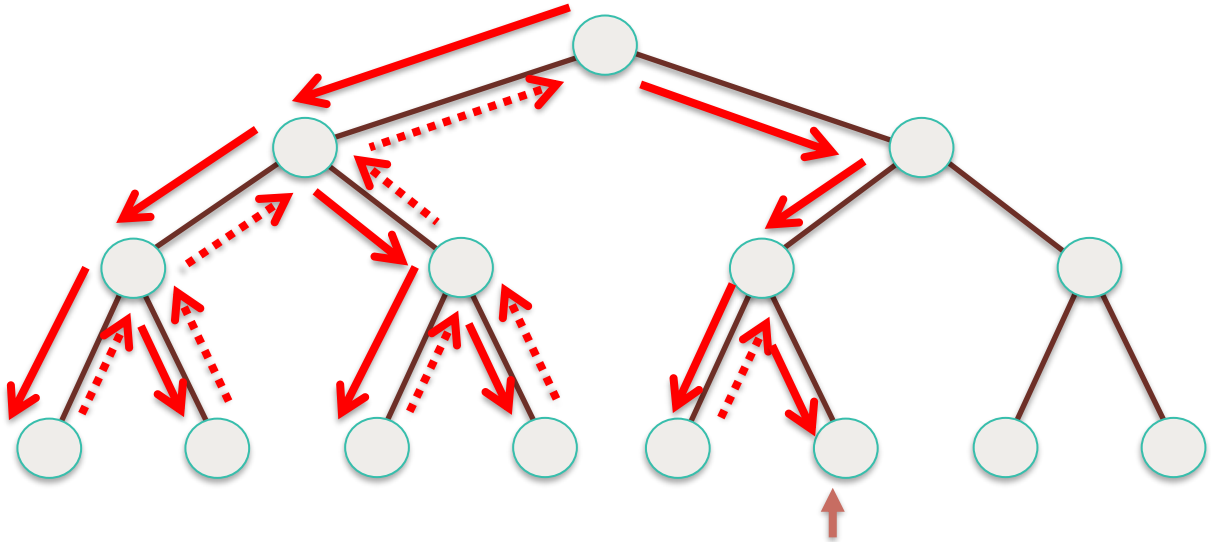
Depth-First Search (DFS): Tree



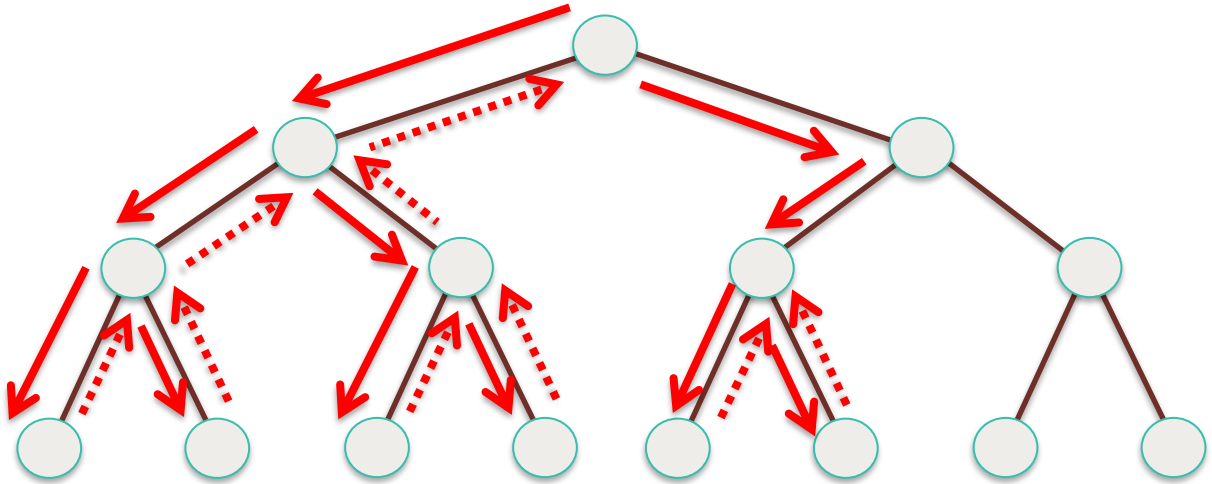
Depth-First Search (DFS): Tree



Depth-First Search (DFS): Tree

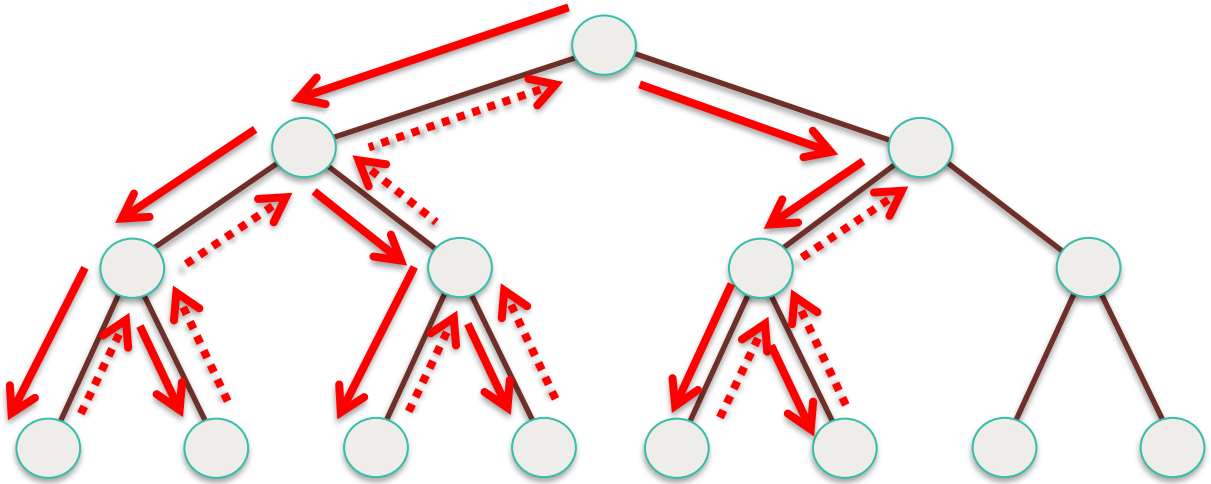


Depth-First Search (DFS): Tree

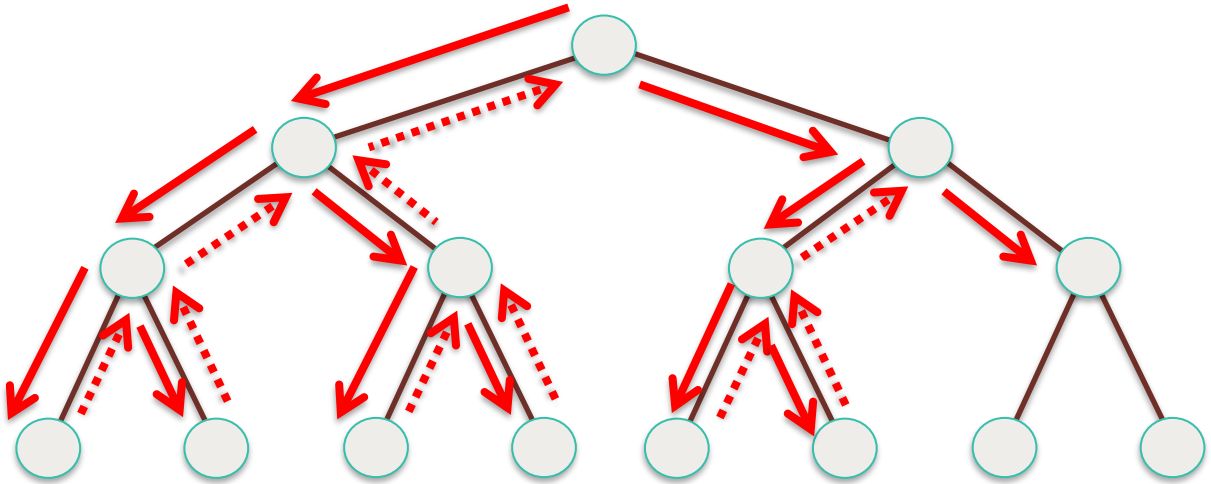


Depth-First Search (DFS): Tree

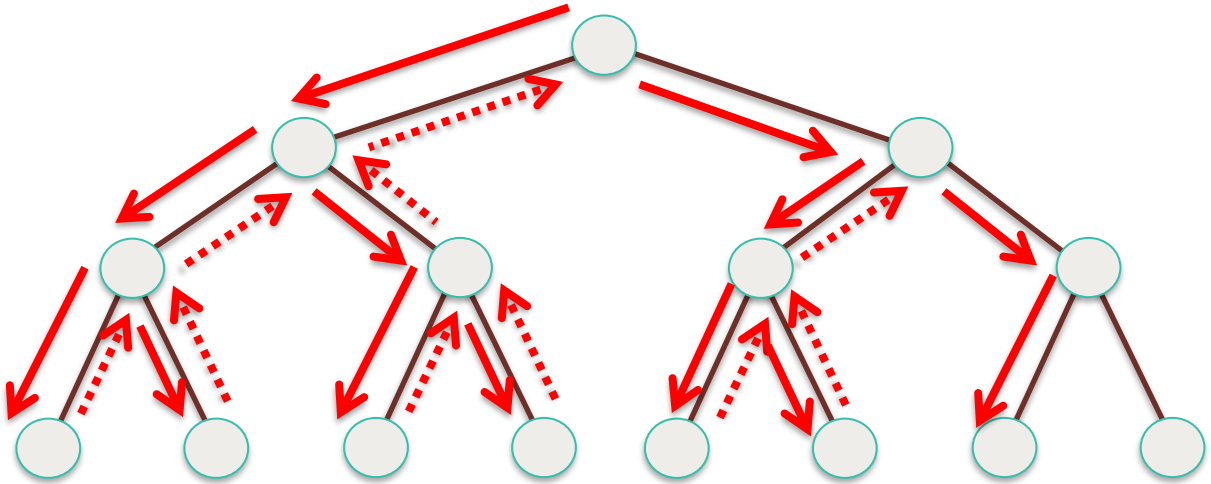
Depth-First Search (DFS): Tree



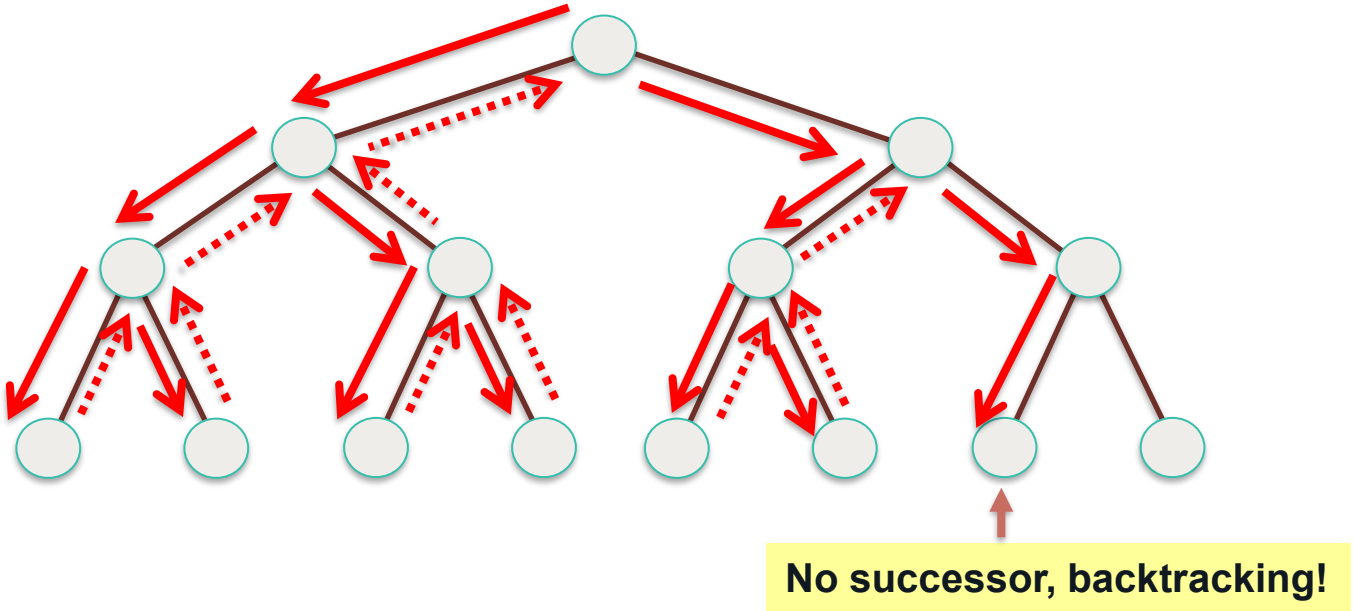
Depth-First Search (DFS): Tree



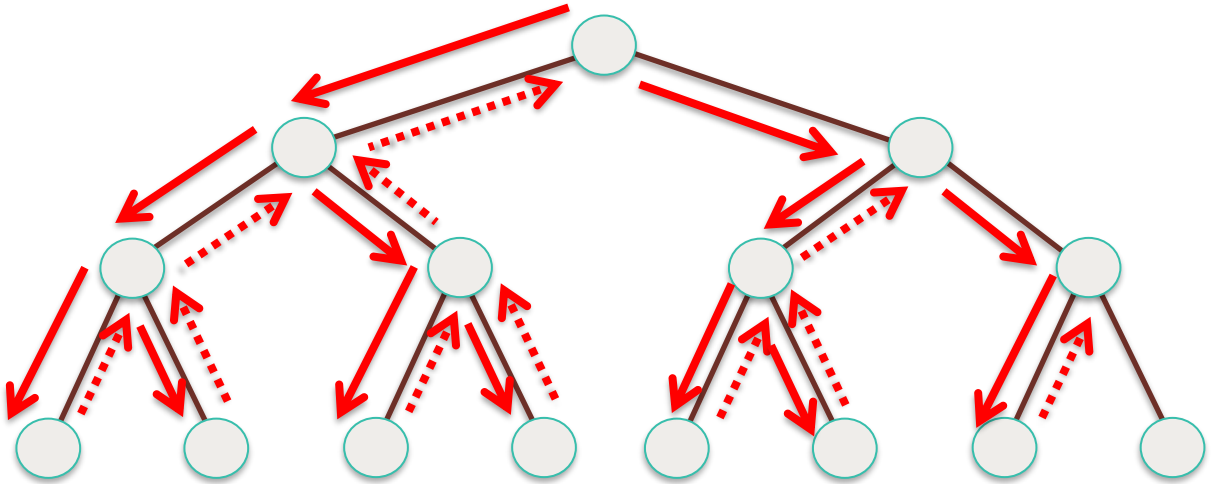
Depth-First Search (DFS): Tree



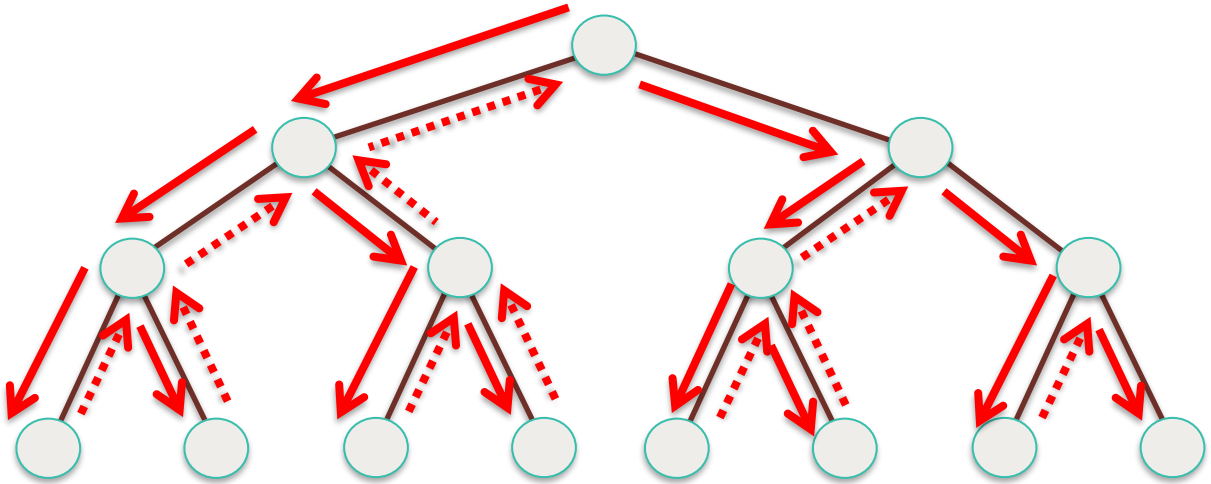
Depth-First Search (DFS): Tree



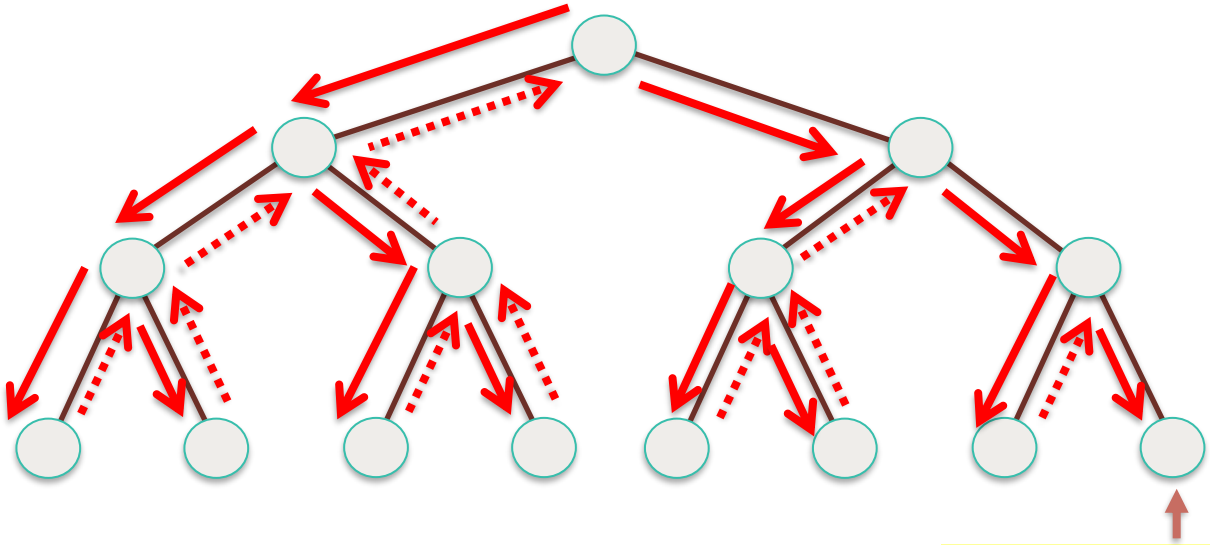
Depth-First Search (DFS): Tree



Depth-First Search (DFS): Tree

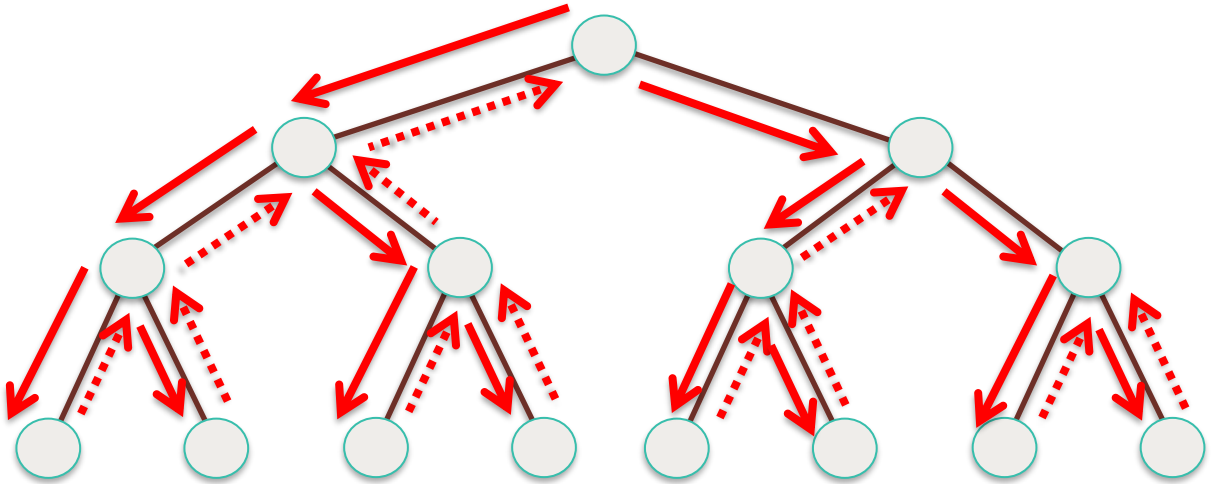


Depth-First Search (DFS): Tree

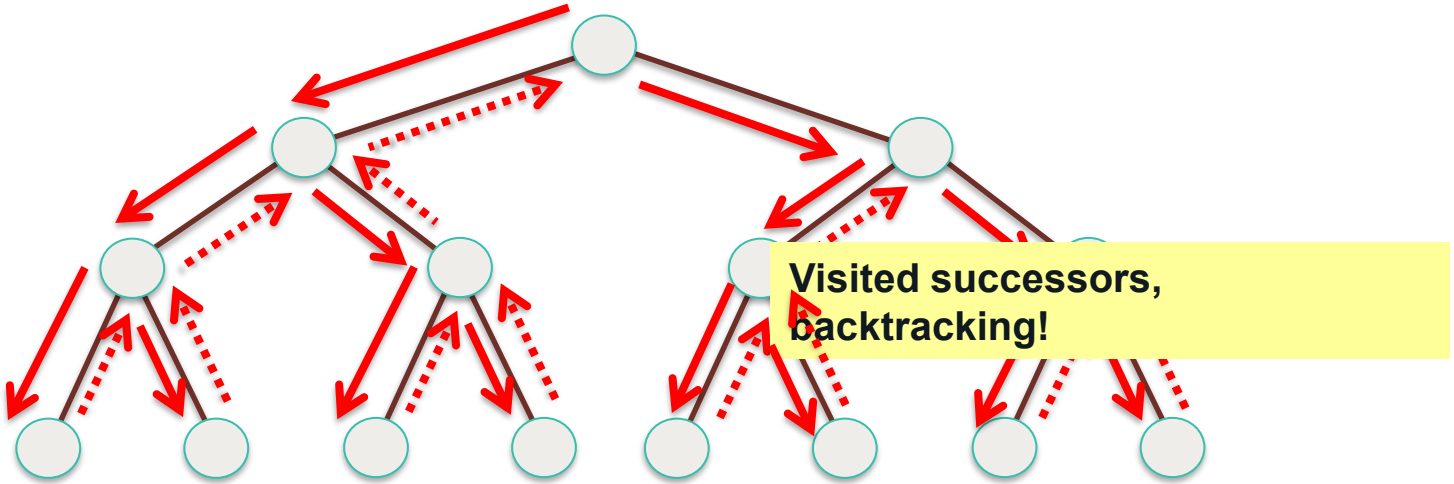


No successor, backtracking

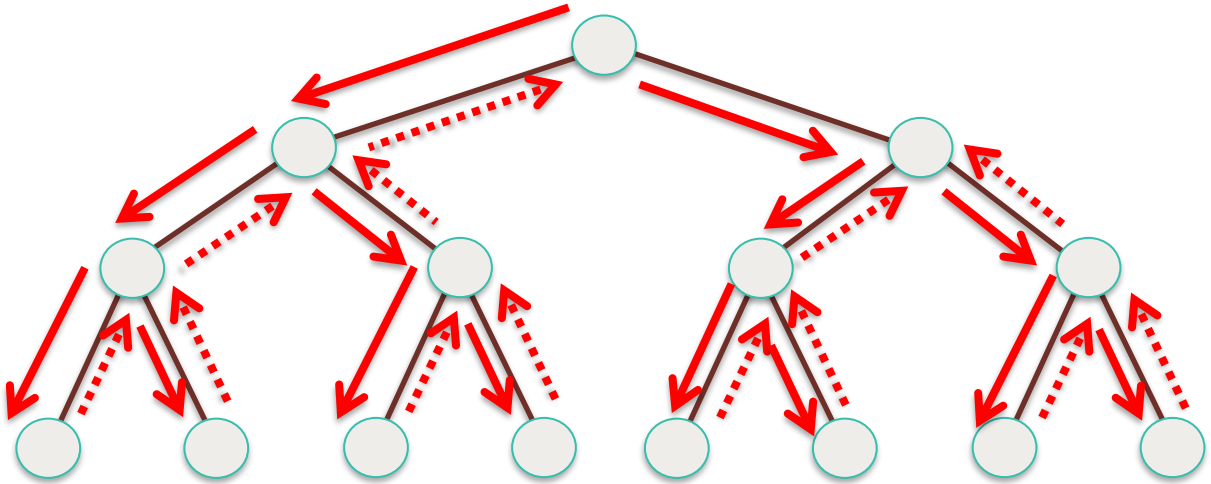
Depth-First Search (DFS): Tree



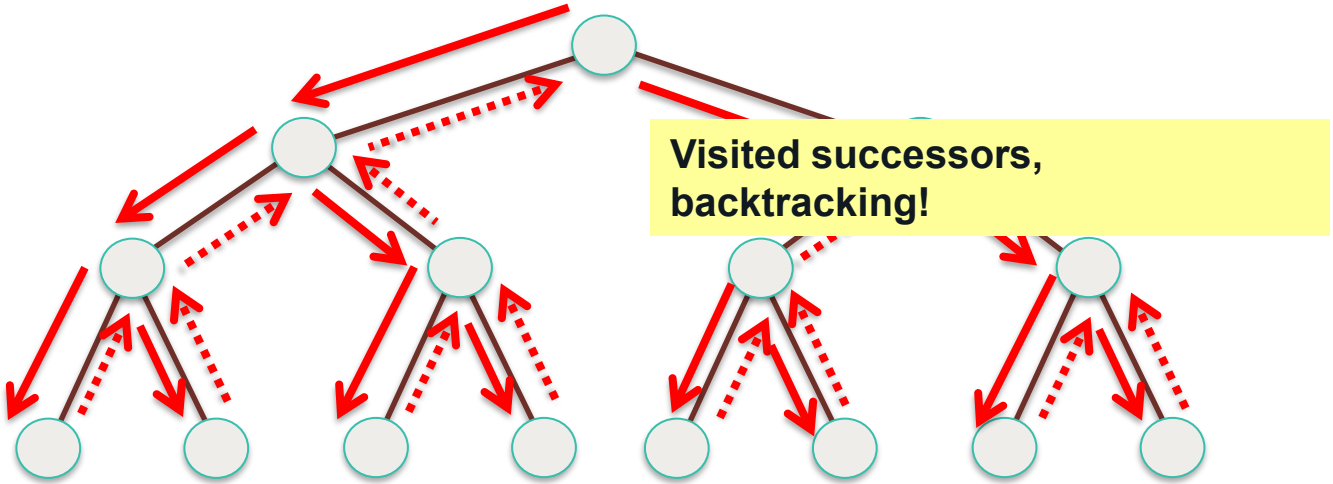
Depth-First Search (DFS): Tree



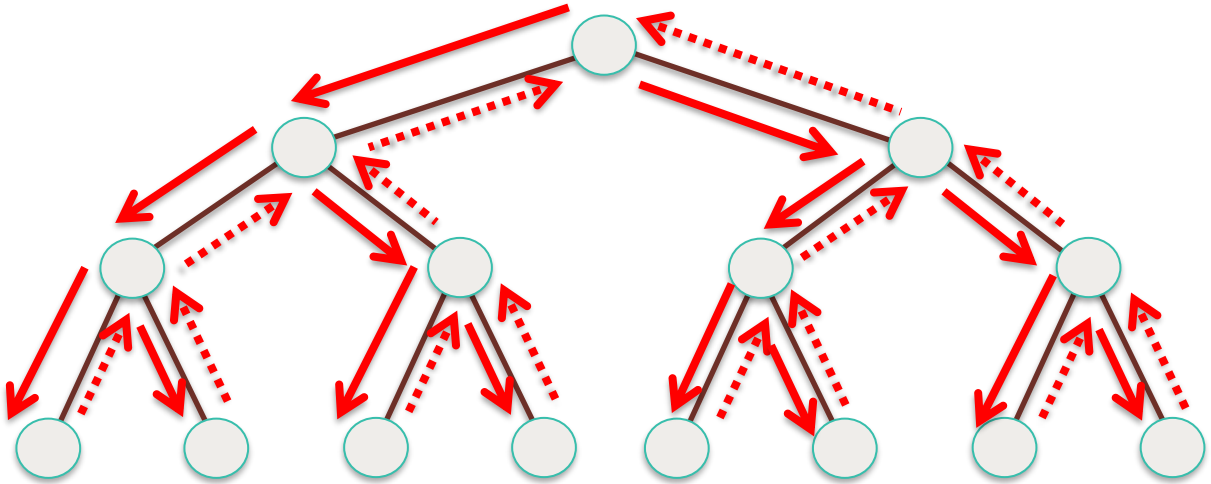
Depth-First Search (DFS): Tree



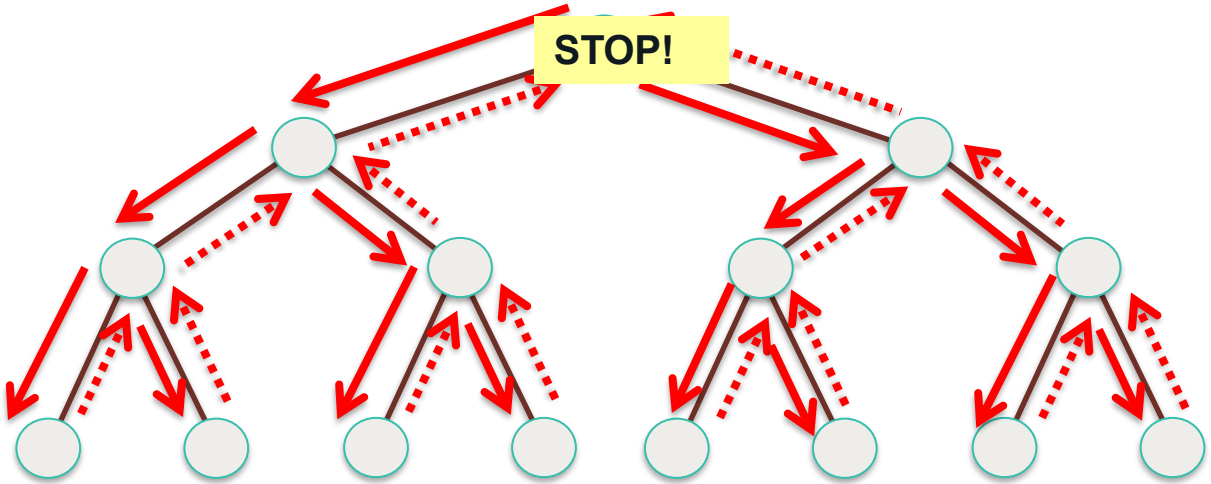
Depth-First Search (DFS): Tree



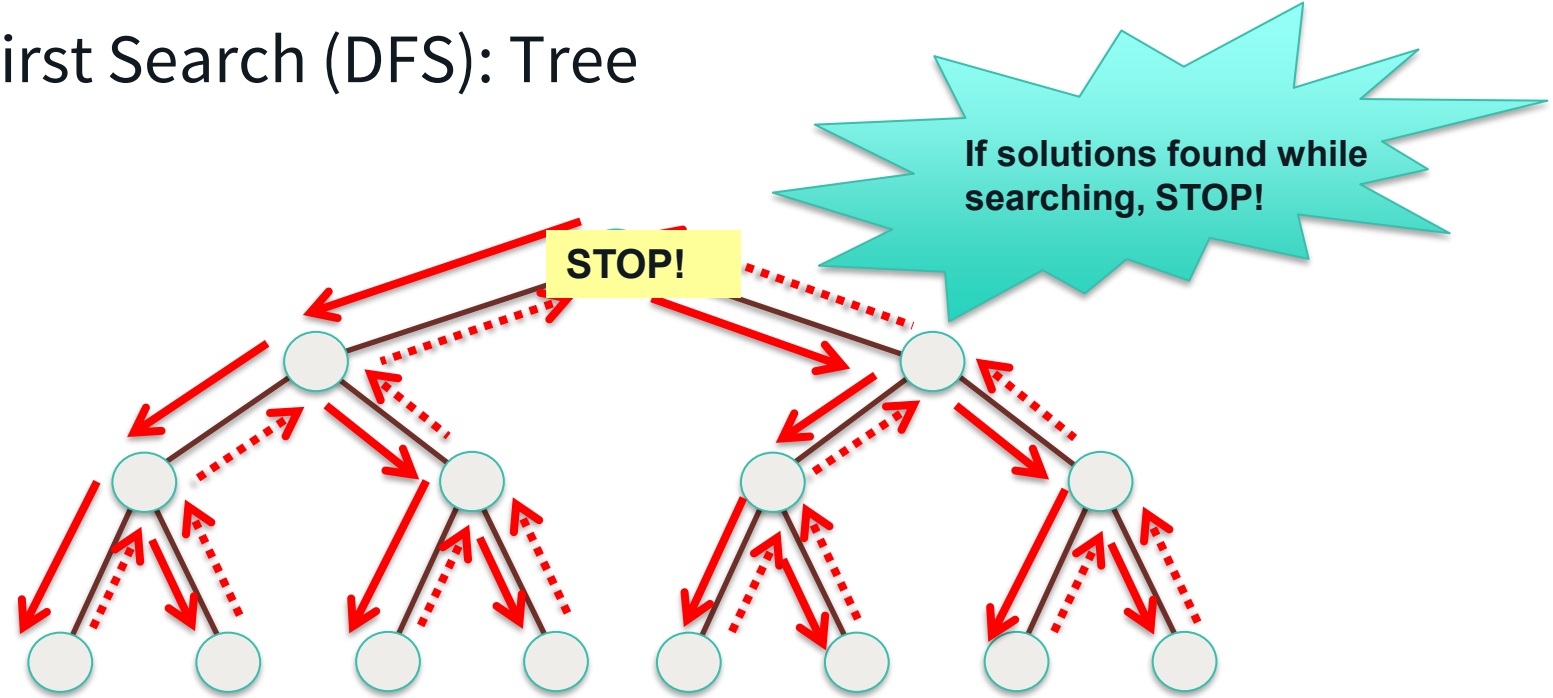
Depth-First Search (DFS): Tree



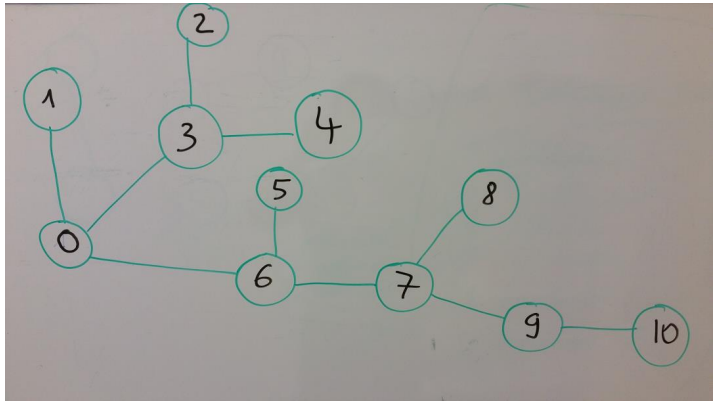
Depth-First Search (DFS): Tree



Depth-First Search (DFS): Tree



DFS



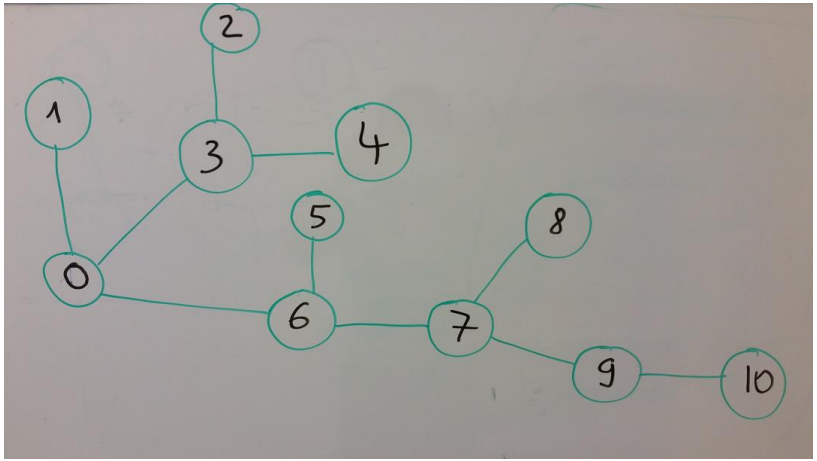
Start vertex (Root): 0 (but can be any other vertex)
Use Stack (Last-in-First-Out). Another name: LIFO queue

DFS Visit List: { 0, 6, 7, 9, 10, 8, 5, 3, 4, 2,1}

Stack (S)?

Step 1: S={0} => Visit 0. Add children of 0 to S.
Step 2: S={1,3,6} => Pick the last element of S i.e, visit 6.
Step 3: S={1,3} => Add all children of 6 to S.
Step 4: S={1,3,5,7}
Step 5: S={1,3,5,7}=> Pick 7 and Visit 7. Add all children of 7 to S.
Step 6: S={1,3,5,8,9}
Step 7: S={1,3,5,8,9}=> Pick 9 and Visit 9. Add all children of 9 to S.
Step 8: S={1,3,5,8,10}
Step 9: S={1,3,5,8,10} => Pick 10 and Visit 10.
Step 8: S={1,3,5,8} => Pick 8 and Visit 8.
Step 9: S={1,3,5} => Pick 5 and Visit 5.
Step 10: S={1,3} => Pick 3 and Visit 3. Add all children of 3 to S.
Step 11: S={1,2,4}
Step 12: S={1,2,4}=> Pick 4 and Visit 4.
Step 13: S={1,2}=> Pick 2 and Visit 2.
Step 14: S={1}=> Pick 1 and Visit 1.
Step 15: S= {∅} **STOP**.

Ideas for implementation

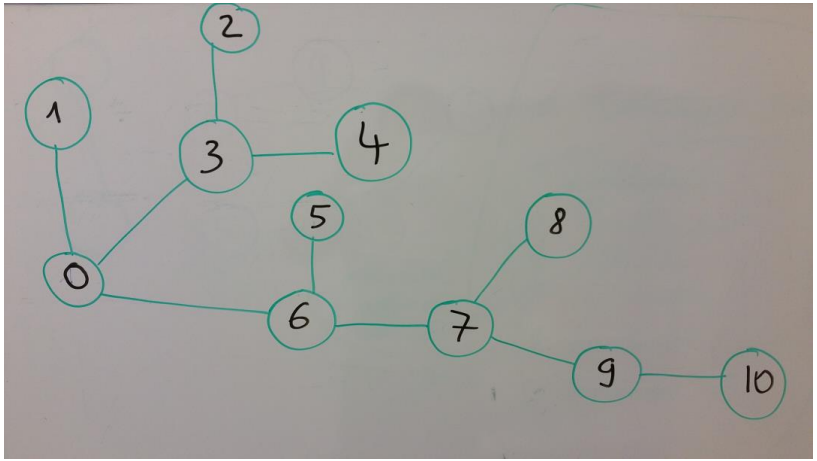


Data structure first: adjacent matrix or adjacent list ?
Start vertex: 0 (but can be any other vertex)
Use Stack (Last-in-First-Out). Another name: LIFO queue

DFS: { 0, 6, 7, 9, 10, 8, 5, 3, 4, 2,1 }

Use add(), addAll(), pop() methods in Stack

Ideas for implementation



Data structure first: adjacent matrix or adjacent list ?
Start vertex: 0 (but can be any other vertex)
Use Stack (Last-in-First-Out). Another name: LIFO queue

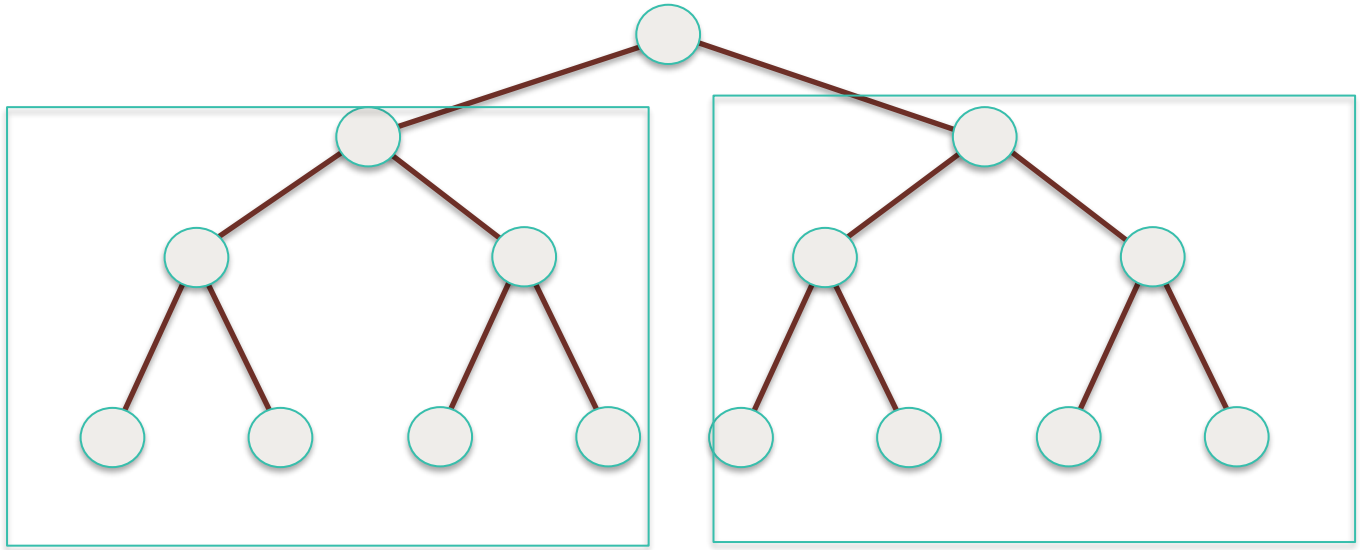
DFS: { 0, 6, 7, 9, 10, 8, 5, 3, 4, 2,1 }

Alternative for implementation
Serial DFS: Use Recursive call

Use add(), addAll(), pop() methods in Stack

Parallelize DFS

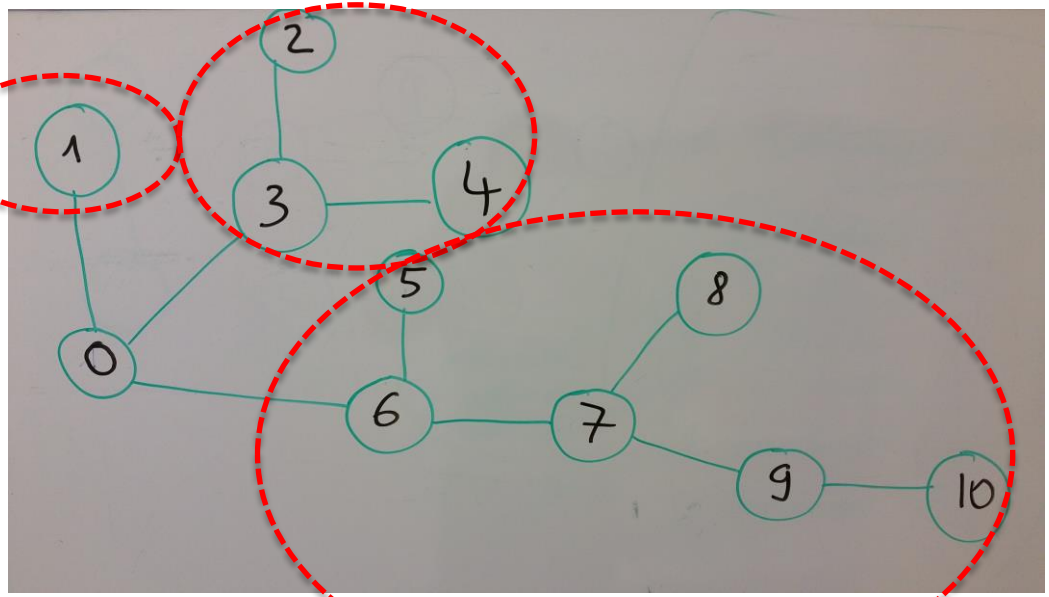
Parallel Depth-First Search



DFS

task

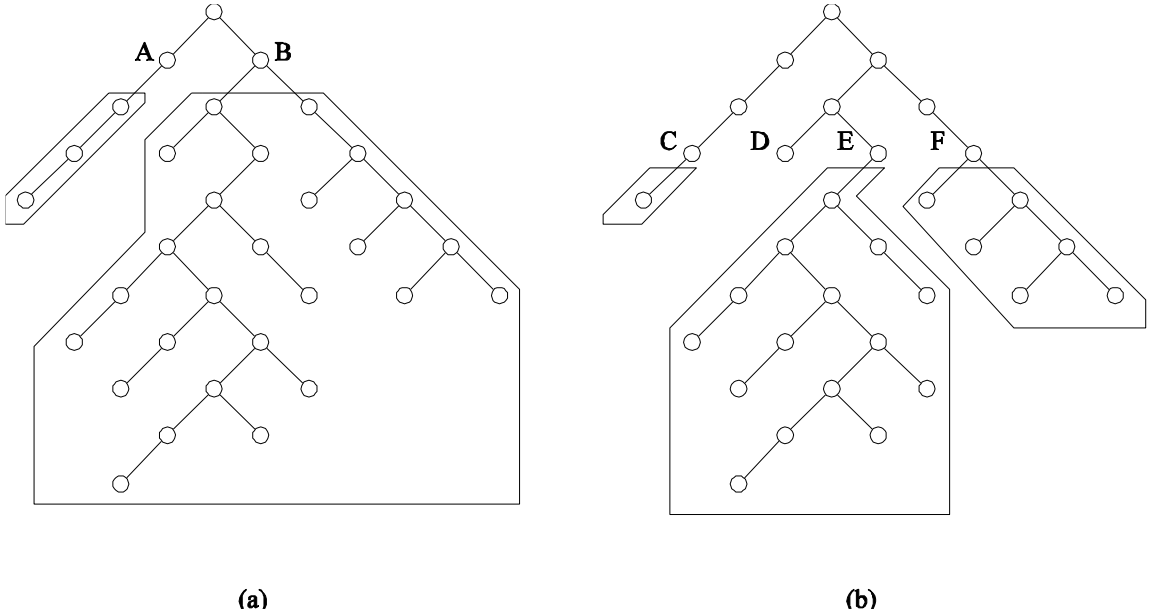
task



task

Submit tasks to an Executor framework

DFS



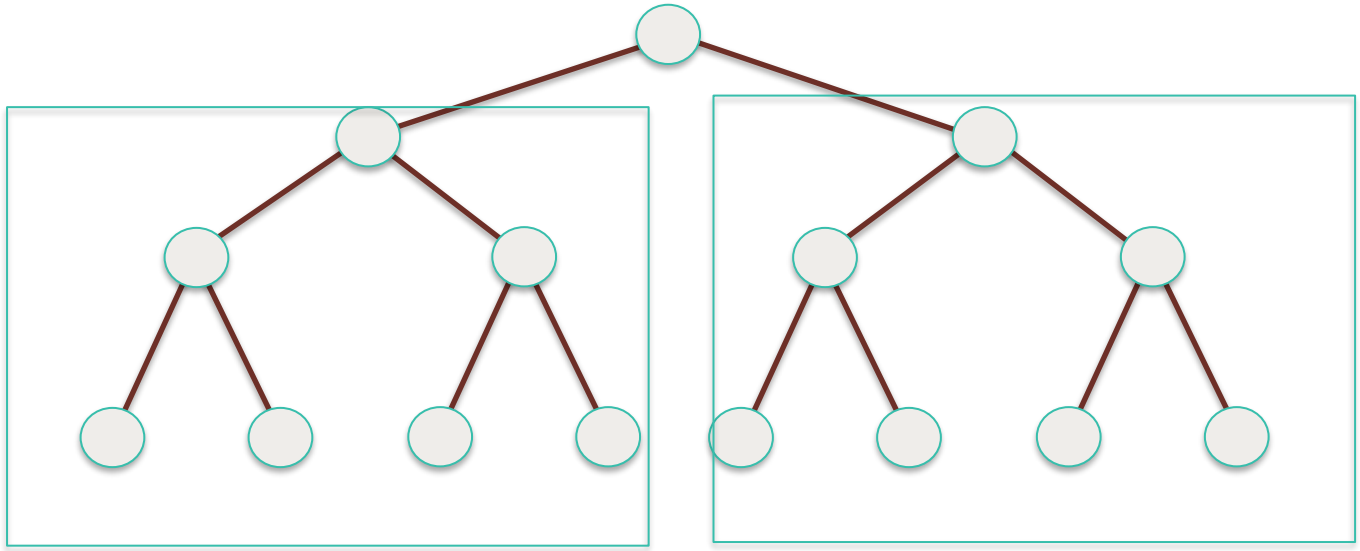
The unstructured nature of tree search and the imbalance resulting from static partitioning

DFS: Dynamic Load Balancing

- When a processor runs out of work, it gets more work from another processor
- This is done using work requests and responses in message passing machines and locking and extracting work in shared address space machines
- On reaching final state at a processor, all processors terminate
- Unexplored states can be conveniently stored as local stacks at processors
- The entire space is assigned to one processor to begin with

Slide adjusted from the one provided by the authors

Parallel Depth-First Search



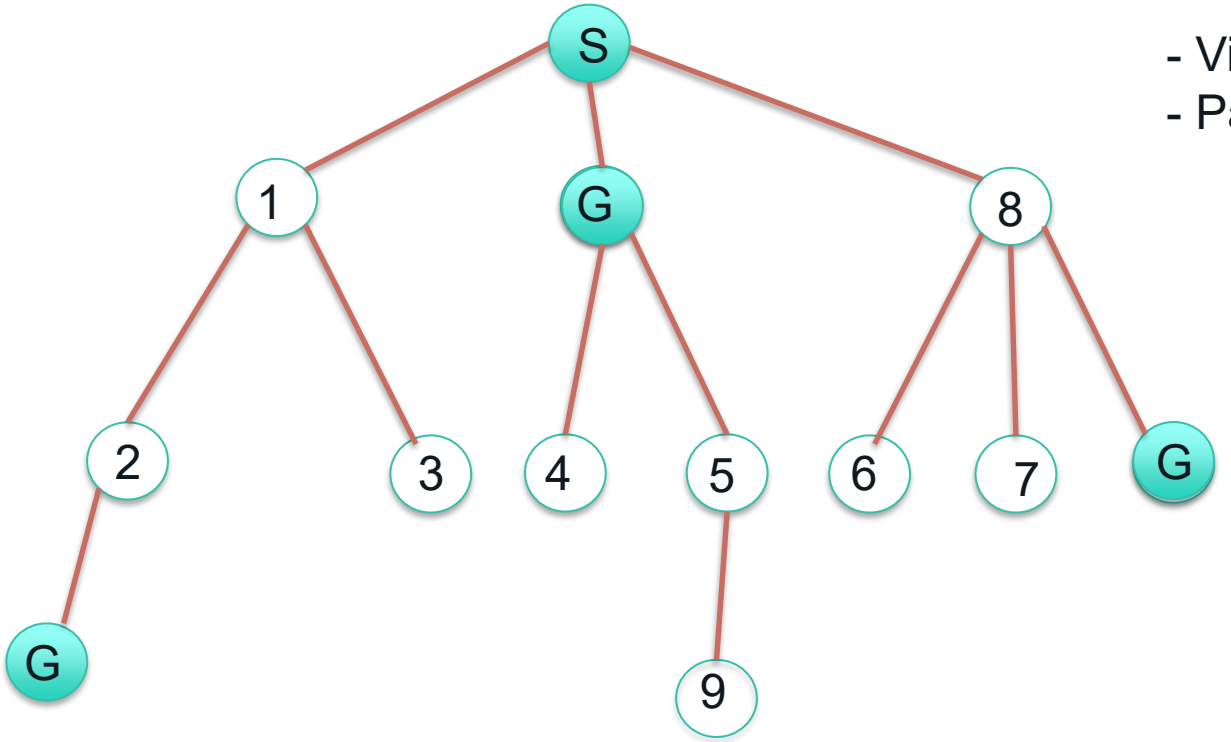
BSF and DFS Parallelism

- How the search space is divided among threads?
- Different subtrees can be searched concurrently
- However, subtrees can be very different in size
- It is difficult to estimate the size of a subtree rooted at a node

Østfold University College

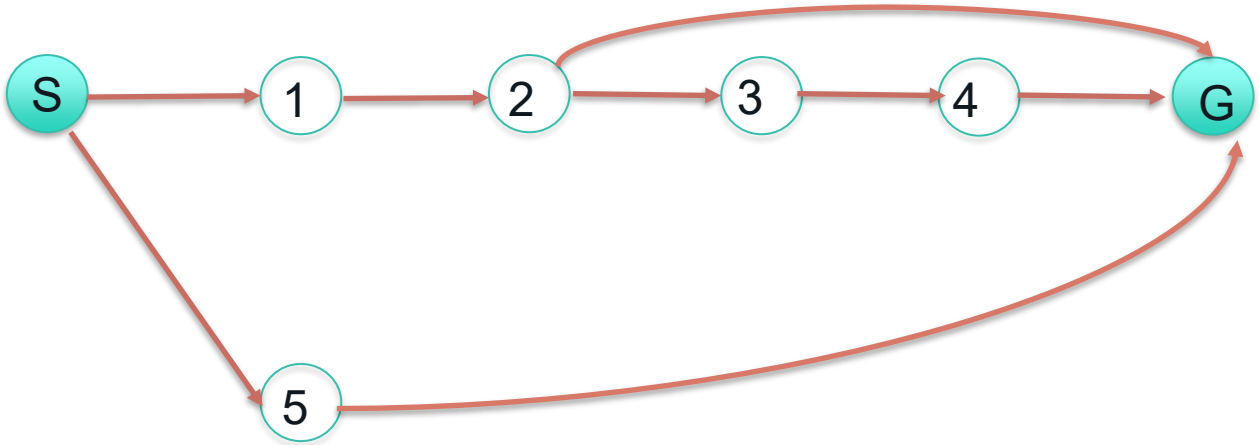
Application of Graph Search in path finding

BFS – Which path can BFS find to move from S to G

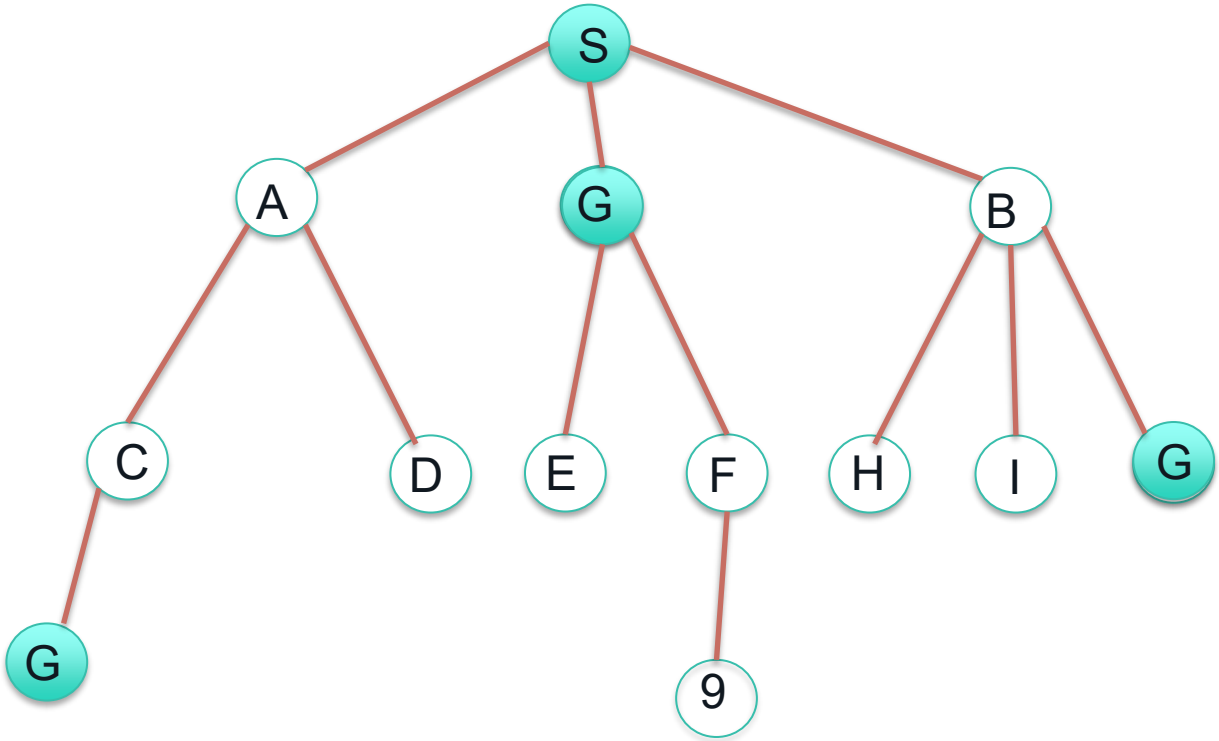


- Visiting list
- Path

BFS – Which path can BFS find to move from S to G



DFS – Which path can DFS find to move from S to G



DFS – Which path can DFS find to move from S to G